

PROJECT REPORT

Digital Design Implementation of the IEEE 802.15.4 Physical Layer (PHY)

RTL Design, Fixed-Point Modeling, Functional Verification, FPGA & ASIC Flow
(From RTL to GDSII)

Authors : Anas Mostafa Abo-Lila

David Sameeh Fares

Robir Tamer Raouff

Mazen Mahmoud Mohammed

Sherif Ahmed

Digital IC Design ITI Summer training 2026

Supervisors: Dr/ Ahmed Shalaby

Eng/ Ahmed El-Gammal

Eng/ Youssef El-Sherif

Table of Contents

- **1. Block Diagram & Quick Demonstration**
 - **1.1 System Block Diagram**
 - **1.2 Quick Demonstration**
- **2. FSM Diagrams**
 - **2.1 Transmit Control FSM**
 - **2.2 Frame Formation FSM**
 - **2.3 CSK Chip Generator FSM**
- **3. Word Length / ROM Design Decisions & MSE Justification**
 - **3.1 Fixed-Point Format Summary**
 - **3.2 Word-Length Selection Rationale**
 - **3.3 MSE Sweep & Justification**
 - **3.4 ROM Sizing & Addressing**
- **4. Verification Methodology & Results**
 - **4.1 Verification Strategy**
 - **4.2 Testbench Architecture**
 - **4.3 Test Cases & Coverage**
 - **A - Case 1, Testing Fast Mode**
 - **B - Case 2, Testing Slow Mode**
 - **C - Case 3.1, Testing Hybrid Mode (Fast)**
 - **D - Case 3.2, Testing Hybrid Mode (Slow)**
 - **4.4 Results Summary**
- **5. FPGA Resource Utilization, Timing Results & Known Limitations**
 - **5.1 Resource Utilization**
 - **5.2 Power Results**
 - **5.3 Timing Results**
 - **5.4 Schematics**
 - **5.5 Implementation**
 - **5.6 Timing Constraints**
 - **5.7 Physical Constraints**
 - **5.8 UART Protocol & Pin Allocation**
- **6. ASIC Resource Utilization, Timing Results & Known Limitations**
 - **6.1 Synthesis**
 - **6.2 Formal Verification**
 - **6.3 Place & Route (PnR)**
 - **6.3.1 NDM**
 - **6.3.2 Design Library**
 - **6.3.3 Floor Plan**
 - **6.3.4 Power Plan**
 - **6.3.5 Placement**
 - **6.3.6 CTS**
 - **6.3.7 Routing & Finishing**

1. Block Diagram & Quick Demonstration

This section presents the overall system architecture and provides a brief walkthrough demonstrating end-to-end functionality.

1.1 System Block Diagram

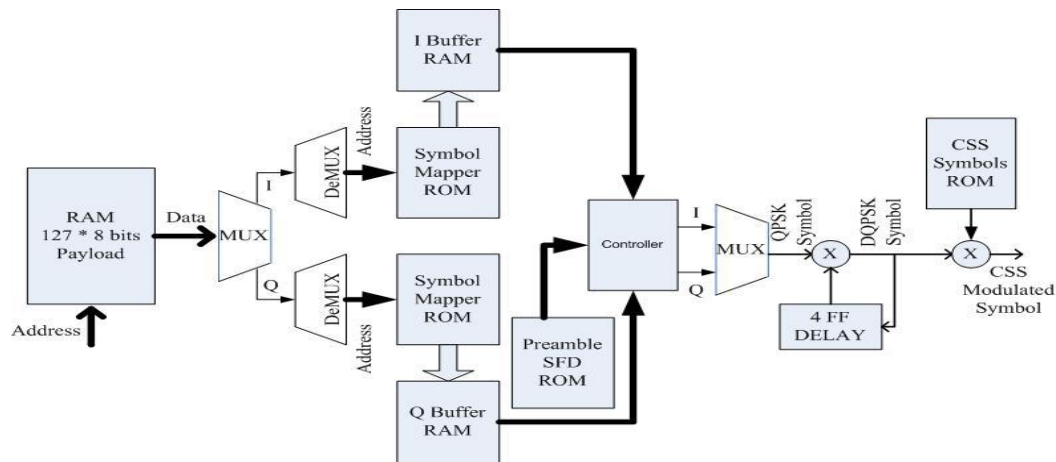


Figure 01-1, Recommended CSS transmitter architecture

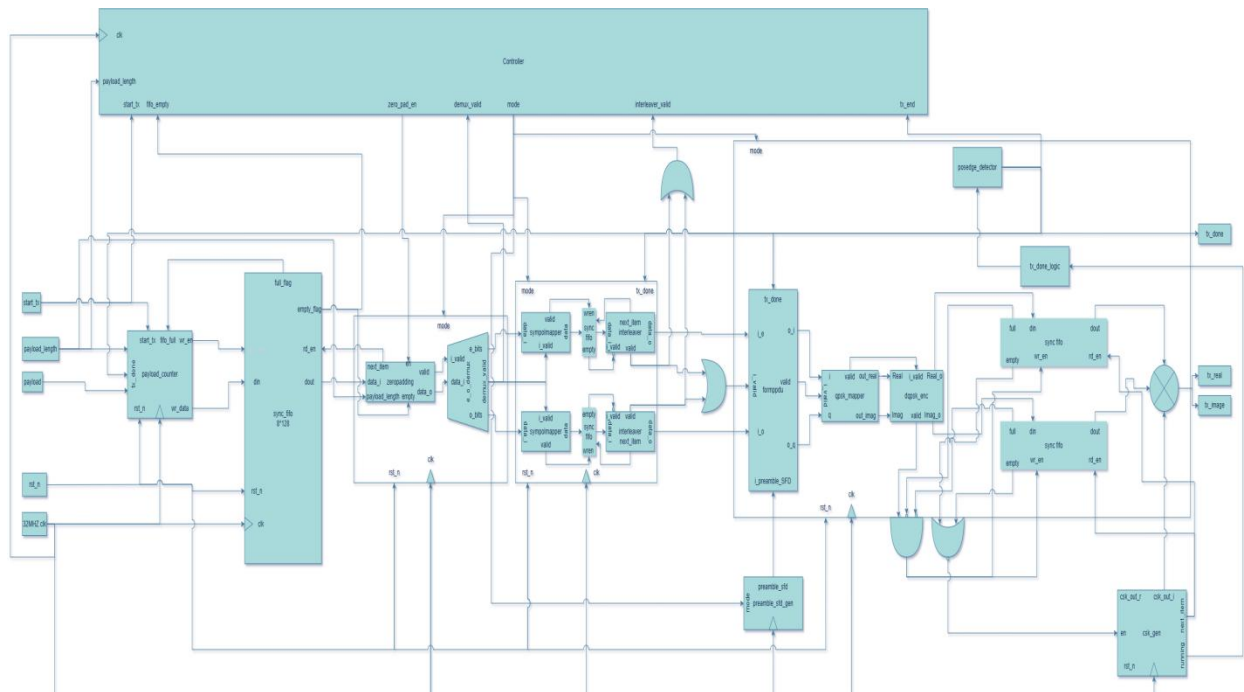


Figure 01-2, Implemented CSS transmitter architecture

1.2 Quick Demonstration

[Insert a short walkthrough: one representative input stimulus, the expected output, and a screenshot or waveform showing the design producing that output. State which data rate(s) and configuration were used.]

A - IEEE 802.15.4 PHY Interface :

Signal	Width	Direction	Description
clk	1	IN	System clock
reset	1	IN	Synchronous system reset
start_Tx	1	IN	Pulse from MAC to PHY to initiate transmission
payloadLength	8	IN	Payload length in bytes (≤ 127)
payload_din	8	IN	Payload data written to the payload RAM before start_Tx
done_Tx	1	OUT	Pulses high when the whole PPDU has been transmitted
Tx_real	8	OUT	Real part of the modulated CSS output sample
Tx_imag	8	OUT	Imaginary part of the modulated CSS output sample

B - Sub modules (micro architecture summary) :

1 - Payload Counter

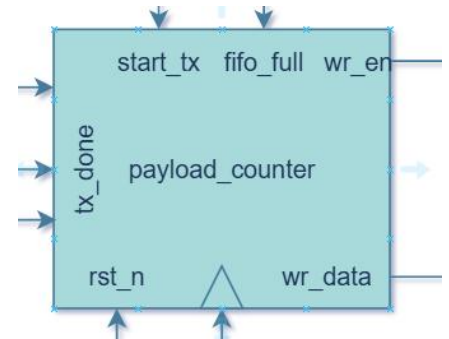
Function: Loads incoming payload bytes into the payload FIFO, one byte per active clock cycle, up to payload_length, and signals completion.

Key Signals:

- Inputs: start_tx, tx_done, fifo_full, payload, payload_length
- Outputs: wr_en, wr_data

Behavior:

1. Idle after reset/tx_done, waiting for start_tx.
2. start_tx arms the module (running).
3. While enabled: writes one payload byte per cycle until counter = payload_length, then stops.
4. Writes pause automatically if fifo_full is asserted.
5. tx_done resets the counter for the next packet.



Data-Rate Independence: No rate_mode parameter — behaves identically for 1 Mb/s, 250 kb/s, and Hybrid modes, since payload loading itself is rate-agnostic; rate-specific processing begins in later stages.

2 - Payload Sync FIFO

Function: Standard synchronous FIFO that buffers payload bytes between the Payload Counter (write side) and the Zero Padding stage (read side), decoupling their timing.

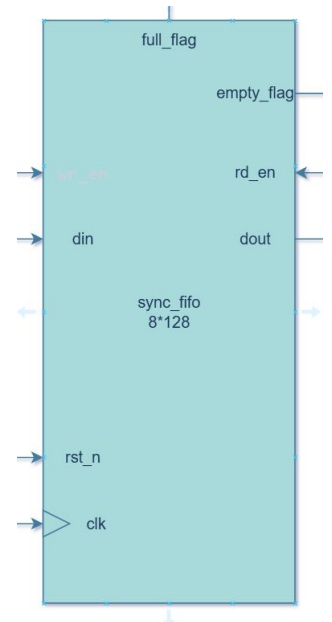
Key Signals:

- Inputs: wr_en, din (from Payload Counter), rd_en (zeropadding_nextitem, from Zero Padding)
- Outputs: dout (to Zero Padding), full_flag (to Payload Counter), empty_flag (to Zero Padding)

Parameters: FIFO_WIDTH = payload_w (8 bits), FIFO_DEPTH = max_payload_length + 1 (128 entries) — sized to hold one full maximum-length payload.

Behavior:

- Writes one byte per cycle when wr_en is high and the FIFO isn't full.
- Reads one byte per cycle when rd_en is high and the FIFO isn't empty.
- full_flag/empty_flag generated via standard MSB-extended read/write pointer comparison.



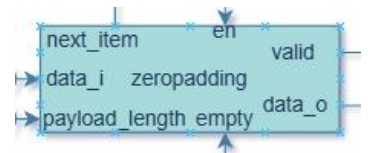
Data-Rate Independence: Same FIFO instance/parameters used regardless of rate_mode — buffering behavior is identical for 1 Mb/s, 250 kb/s, and Hybrid.

3 - Zero Padding

Function: Builds the 12-bit PHY header (PHR = 7-bit payload length + 5 reserved bits), serializes it followed by the payload bytes (MSB-first), and pads the total stream with trailing zeros so its length is a multiple of the data-rate-specific block size (6 bits for 1 Mb/s, 24 bits for 250 kb/s) — matching Symbol Mapper input requirements downstream.

Key Signals:

- Inputs: data_i (payload byte from FIFO), payload_length, empty (payload FIFO empty flag), en (from Controller), mode (rate select, Hybrid only)
- Outputs: valid, data_o (serial bit stream), next_item (read request to payload FIFO)



Behavior (per mode):

1. **Header phase:** Serializes payload_length into a 12-bit PHR, output bit-by-bit (LSB first via right-shift register), pulsing next_item near the end to prefetch the first payload byte.
2. **Payload phase:** Serializes each payload byte MSB-first, requesting the next byte from the FIFO after every 6th bit consumed (leaves 2 header/alignment bits before next byte load).
3. **Padding phase:** After all payload bits are sent, continues outputting zero bits until the total serialized length is a multiple of the required block size (6 for 1 Mb/s, 24 for 250 kb/s) — tracked via counter.

- Handles `payload_length == 0` as a special case (padding starts immediately after the header, no payload phase).

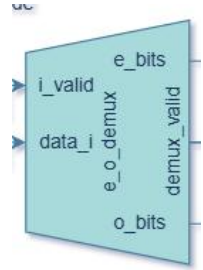
Mode Dependence: Three parallel implementations (F/S/H generate branches) — differ only in the padding block size (6 vs 24 bits) and, in Hybrid mode, select between them via mode.

4 - Even/Odd Demux (e_o_demux)

Function: Splits the serial zero-padded bit stream into two parallel paths — even-indexed bits (I/Real) and odd-indexed bits (Q/Imag) — accumulating each into a parallel group (3 bits for 1 Mb/s, 6 bits for 250 kb/s) before releasing both groups together for the Symbol Mapper.

Key Signals:

- Inputs: `data_i` (serial bit from Zero Padding), `valid_i`, `mode` (rate select, Hybrid only)
- Outputs: `e_bits`, `o_bits` (parallel groups), `valid`



Behavior:

- An internal odd toggle flip-flop alternates every valid input bit, routing it into either `e_bits_reg` (even) or `o_bits_reg` (odd) via a shift-in accumulator.
- Each path counts its own accumulated bits (`e_counter/o_counter`); once both counters reach the required group size (3 or 6, wrapping via modulo-style counting), the accumulated groups are latched to `e_bits/o_bits` and `valid` pulses high for one cycle.
- If `valid_i` drops, counters and the odd toggle reset — resynchronizing on the next valid stream.

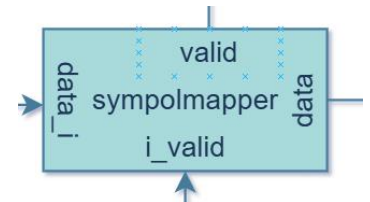
Mode Dependence: Three parallel implementations (F/S/H) — differ only in group width (3 vs 6 bits) and counter wrap value; Hybrid selects behavior via mode.

5 - Symbol Mapper (symbol_mapper)

Function: Converts each parallel data symbol from the Demux stage into its bi-orthogonal codeword — 3-bit symbol → 4-chip codeword for 1 Mb/s, or 6-bit symbol → 32-chip codeword for 250 kb/s — via direct combinational-style lookup, registered on the clock edge.

Key Signals:

- Inputs: `i_data` (3-bit or 6-bit parallel symbol, width set by `rate_mode`), `i_valid`, `mode` (rate select, Hybrid only)
- Outputs: `o_data` (4-bit or 32-bit codeword), `o_valid` (registered copy of `i_valid`, one cycle later)



Behavior:

1. **1 Mb/s (F):** the MSB of the 3-bit symbol (`i_data[2]`) selects sign (bitwise invert or not), while the lower 2 bits (`i_data[1:0]`) select one of 4 base 4-chip patterns via case — implementing the [W; -W] bi-orthogonal structure (positive Hadamard row vs. its complement) with a compact 4-entry table instead of a full 8-row ROM.
2. **250 kb/s (S):** a full 64-entry case statement directly maps every possible 6-bit symbol value to its complete 32-chip codeword — the entire Hadamard [W; -W] table (32 positive rows + 32 negated rows) hard-coded as literal bit patterns rather than computed.
3. **Hybrid (H):** combines both behaviors under one instantiation, selected at runtime by mode: mode=0 reproduces the full 250 kb/s 64-entry table on the 32-bit output; mode=1 reproduces the 1 Mb/s 4-entry sign/pattern lookup, but zero-extends the result into the same 32-bit `o_data` width (writing only `o_data[3:0]`, forcing `o_data[31:4]=0`) so both rates share one consistently-sized output port.

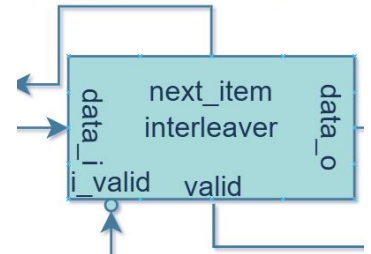
Mode Dependence: Three parametrized implementations (F/S/H) selected at instantiation via `rate_mode`; F and S are rate-fixed (width and table hard-set at compile time), while H is the single dual-rate variant, runtime-switched via the mode input, matching the pattern established in `e_o_demux`.

6 - Interleaver (interleaver)

Function: Serializes the Symbol Mapper's parallel codeword output into a single-bit chip stream (chip 0 transmitted first); for 250 kb/s it additionally performs the spec-mandated 6.5a.2.9 permutation across two consecutive codewords before serializing, while 1 Mb/s bypasses interleaving entirely.

Key Signals:

- Inputs: `i_data` (4-bit or 32-bit codeword from Symbol Mapper), `i_valid_c`, mode (rate select, Hybrid only), `tx_done` (resets the interleaving pair-state between frames)
- Outputs: `o_data` (single serial chip bit), `o_valid`, `next_item` (FIFO read-enable, requesting the next codeword)



Behavior:

1. **1 Mb/s (F):** no interleaving — a 4-bit codeword is loaded into a shift register and shifted out MSB-first over 4 cycles (`o_data <= i_data[3]` first, then 3 more shifted bits), driven continuously since `next_item` is held high every cycle.
2. **250 kb/s (S):** accumulates **two consecutive** 32-bit codewords (`shift_reg[31:0]` = first codeword, `i_data` = second, arriving one FIFO-read cycle later) into a 64-bit register, splits both into 16 four-chip groups (`g0–g15`), and re-packs them per the spec's fixed group order `G0,G13,G2,G15,G4,G9,G6,G11,G8,G5,G10,G7,G12,G1,G14,G3` before shifting the permuted 64-bit result out serially, MSB-first, over 64 cycles. A `first_done/data_ready/cycle_flag` handshake sequence manages the two-codeword accumulation and issues `next_item` pulses at the correct cycles to keep the FIFO's registered read latency synchronized with when each codeword is actually needed.
3. **Hybrid (H):** both behaviors combined in one instantiation, switched at runtime via mode (mode=1 → 1 Mb/s bypass path, mode=0 → 250 kb/s interleaver path) — structurally identical logic to F/S respectively, just folded into shared registers under one always block.

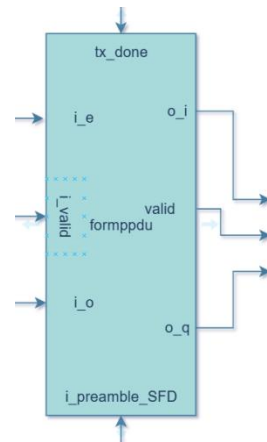
Mode Dependence: Same three-variant pattern as `e_o_demux/symbol_mapper` (F/S/H via `rate_mode` parameter, H runtime-switched via `mode`). Notably, this file's own header documents four specific bugs it corrects relative to an earlier version — undeclared 1-bit group-tap wires causing incorrect concatenation width, a premature `next_item` deassertion causing the second codeword to duplicate the first (FIFO read-latency mismatch), a wrong shift-register tap for the first output bit, and a reversed shift direction/tap in the Hybrid slow path — all now fixed and cross-referenced against the S variant as the corrected reference.

7 - Form PPDU (`form_ppdu`)

Function: Assembles the final PPDU frame by prepending the fixed Preamble+SFD (SHR) header to the interleaver's serial chip stream, applying the identical header to both the I and Q branches per 6.5a.2.5, while delaying the payload chips so they emerge immediately after the SHR window with no gap, duplication, or loss.

Key Signals:

- Inputs: `i_e/i_o` (serial chip from Interleaver's even/odd, i.e. I/Q, paths), `i_valid`, `i_preamble_SFD` (pre-generated SHR pattern, chip 0 in bit 0), `mode` (rate select, Hybrid only), `tx_done` (frame-end/re-arm signal)
- Outputs: `o_i`, `o_q` (final serial chips to the QPSK Mapper), `o_valid`



Behavior:

1. A **3-state FSM** (`ST_IDLE` → `ST_SHR` → `ST_DATA`) governs framing: `ST_IDLE` waits for the first valid chip and immediately emits SHR chip 0; `ST_SHR` shifts out the remaining `shr_len-1` SHR bits (48 for 1 Mb/s, 96 for 250 kb/s) from a right-shifting `reg_shr`; `ST_DATA` then switches the output to the delayed payload stream.
2. In parallel, every incoming interleaver chip (`i_e/i_o/i_valid`) is continuously pushed into a `shr_bits`-deep shift register (`reg_data_i/reg_data_q/reg_valid`) starting from the very first cycle — including during SHR transmission — so that by the time the FSM reaches `ST_DATA`, the payload chip that arrived exactly `shr_len` cycles earlier is sitting at the tap position (`reg_data_i[47]` for 1 Mb/s, [`shr_bits-1`] for 250 kb/s) and gets output with zero gap or overlap relative to the SHR.
3. On `tx_done`, the FSM resets to `ST_IDLE` and the entire delay line/valid shift register is explicitly flushed, preventing any leftover chip from one frame bleeding into the next frame's SHR window.

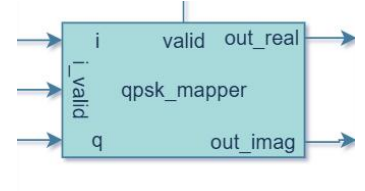
Mode Dependence: Single unified implementation (not three separate generate branches like the earlier modules) — rate selection is done via a `fast_sel` wire (hardwired for F/S, runtime mode input for H) that parametrizes `shr_len` and the delay-line tap position, rather than duplicating the FSM logic per rate.

8 - QPSK Mapper (QPSK_mapper)

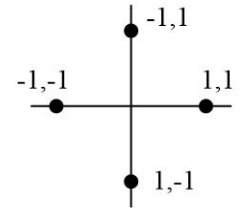
Function: Converts one bit from the I path and one bit from the Q path (both bipolar chip bits from Form PPDU) into a single complex QPSK constellation symbol, matching this project's established formula $X = ((I+Q) - j(I-Q))/2$.

Key Signals:

- Inputs: i (In-phase bit), q (Quadrature bit), i_valid
- Outputs: out_real, out_imag (2-bit signed, values restricted to {-1, 0, +1}), valid



Behavior: A single registered 4-entry case lookup on {i,q} directly produces the correct constellation point every cycle a valid bit pair arrives — {1,1}→(+1,0), {1,0}→(0,-1), {0,1}→(0,+1), {0,0}→(-1,0) — exactly matching the four QPSK corners established earlier for this design, and implemented as a pure combinational-style truth table (no real multiplier/adder hardware), consistent with the "QPSK output is always exactly $\pm 1/0$, so implement as case-statement" principle established for this project. valid mirrors i_valid one cycle later (deasserts when no valid input is present), and reset clears both outputs and valid to a known idle state.



QPSK Constellation diagram

QPSK LUT :

Input from PPDU	Output from QPSK Mapper	Output form
i= 1, q= 1	real= 1, imaginary= 0	$X = 1 + j*(0)$
i= 1, q= 0	real= 0, imaginary= -1	$X = 0 + j*(-1)$
i= 0, q= 1	real= 0, imaginary= 1	$X = 0 + j*(1)$
i= 0, q= 0	real= -1, imaginary= 0	$X = -1 + j*(0)$

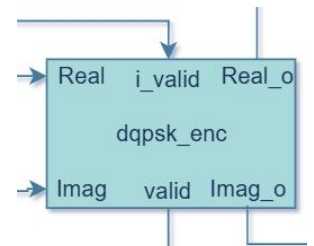
Mode Dependence: None — this module is entirely rate-independent; the QPSK mapping formula is identical for 1 Mb/s and 250 kb/s (only the *rate* of arriving I/Q bit pairs differs upstream, not the mapping itself), so unlike the preceding modules there is no rate_mode/mode parameterization here.

9 - DQPSK Encoder (DQPSK)

Function: Implements the differential encoder (6.5a.2.6): $S_n = X_n \times S_{\{n-4\}}$, where each incoming QPSK symbol (Real, Imag from the QPSK Mapper) is multiplied by the symbol from 4 positions earlier in the stream, held in a 4-deep feedback shift register, reset to the spec-mandated $\exp(j \cdot \pi/4)$ (unnormalized as 1+1j) at the start of each frame.

Key Signals:

- Inputs: Real, Imag (2-bit signed QPSK symbol, X_n), i_valid
- Outputs: Real_o, Imag_o (differentially-encoded symbol, S_n), valid



Behavior:

1. Two 4-element register arrays (Real_FF[0:3], Imag_FF[0:3]) implement the Z^{-4} feedback delay line, all initialized to +1 on reset — matching $\exp(j\pi/4)$ per the spec's initial-condition requirement.
2. On each valid input, the complex multiply $S_n = X_n \cdot S_{n-4}$ is computed directly against Real_FF[3]/Imag_FF[3] (the oldest/4-cycles-ago tap) using the standard complex-multiply expansion $(ac-bd) + j(ad+bc)$, registered onto Real_o/Imag_o the same cycle valid asserts.
3. The **same freshly-computed result** is simultaneously shifted into the front of both delay chains $\{FF[3], FF[2], FF[1], FF[0]\} \leftarrow \{FF[2], FF[1], FF[0], \text{new_result}\}$, so the newly-produced S_n becomes the future S_{n-4} feed exactly 4 valid-cycles later — this is the serial/streaming hardware realization of the algorithm's $\text{feedback_memory} = S_n(i:i+3)$ update established earlier for the golden MATLAB model, just done one symbol per cycle instead of in batches of 4.

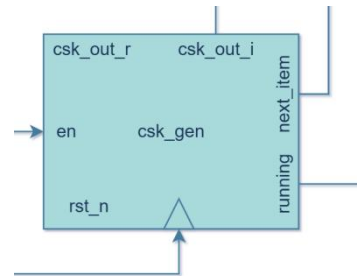
Mode Dependence: None — rate-independent, identical differential-encoding math regardless of 1 Mb/s or 250 kb/s (only the arrival rate of symbols differs upstream, same as the QPSK Mapper).

10 - CSK Generator (CSK_GEN)

Function: Generates the chirp-shift-keying waveform for chirp index $m=1$ — 152 samples of pre-computed chirp $\times$ raised-cosine amplitude (4 subchirps $\times$ 38 samples, Q1.4 fixed-point), followed by an alternating 10-cycle even-gap / 70-cycle odd-gap silence period between successive chirp symbols — and hands the result to the downstream complex multiplier.

Key Signals:

- Inputs: en (fetch/run enable, tied to FIFO non-empty status upstream), clk, rst_n
- Outputs: csk_out_r/csk_out_i (chirp sample, Q1.4), next_item (FIFO read-enable pulse), running (frame-active status flag)

**Behavior:**

1. **ROM implemented as two purely combinational case statements** (rom_r, rom_i) indexed by addr_ptr, rather than a const/initial-block array — the header comment states this was done specifically for **Formality (equivalence-checking tool) compatibility**, avoiding const-array or undriven-net constructs that Formality can't reliably match against a synthesized netlist.
2. **Control FSM:** an internal running latch starts when en first asserts and can only drop when en is low *and* the frame is at a completed boundary (stop_now — checks addr_ptr>151 and the gap counter having reached its terminal value), preventing a trailing gap from being cut short if en drops mid-transmission. first_fetch/boundary_d together implement the previously-established one-cycle-registered pipeline alignment between next_item's FIFO-read pulse and the corresponding ROM sample becoming valid, keeping the DQPSK symbol and chirp sample in lockstep at every subchirp boundary (addresses 37, 75, 113, 151).
3. Address counter (addr_ptr) walks 0→151 through the active region (152 cycles), then the gap branch holds output at zero for 10 or 70 cycles (alternating via which_gab) before resetting addr_ptr and beginning the next chirp symbol.

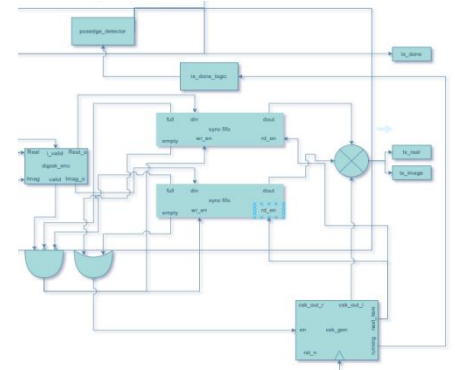
Mode Dependence: None — rate-independent; this module operates purely on the fixed chirp-index-1 waveform and gap timing, identical regardless of 1 Mb/s or 250 kb/s (matching the earlier-established fact that data rate never affects the CSK generator).

11 - DQPSK/CSK Top Wrapper (dqpsk_csk_top)

Function: Integrates the DQPSK differential encoder, two Sync FIFOs (I/Q rate buffering), the CSK Generator, and the final complex multiplier that produces the modulated CSS output (tx_real, tx_imag), plus a delayed, edge-detected tx_done completion signal — this is the exact module previously reviewed in this project's extended verification session (formerly named Top), confirmed here byte-for-byte identical in structure.

Key Signals:

- Inputs: Real/Imag (QPSK/DQPSK symbol stream), i_valid
- Outputs: tx_real/tx_imag (8-bit signed, Q3.4, final CSS output samples), tx_done



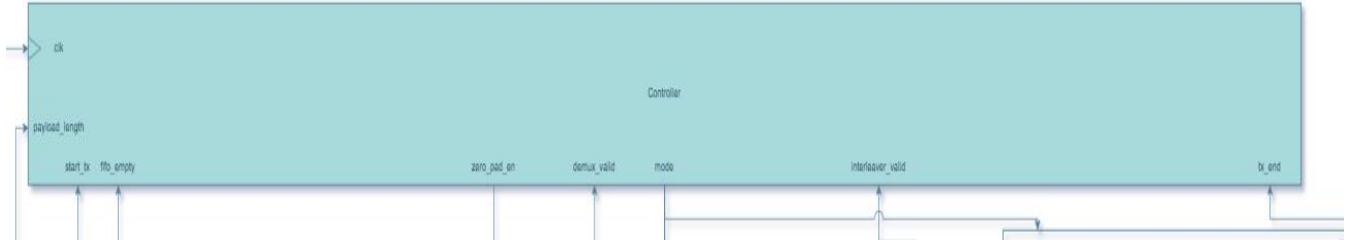
Behavior:

1. **DQPSK → dual FIFOs:** DQPSK_o output feeds two parallel synchronous FIFOs (real and imaginary paths), each write-gated by valid && !full_flag, decoupling the DQPSK's symbol-arrival rate from the CSK Generator's fixed 38-cycle-per-symbol consumption rate.
2. **en generation:** en = !empty_flag_imag & !empty_flag_real — the CSK Generator only runs while both FIFOs have data, matching the earlier-established design where FIFO emptiness (not raw i_valid) gates the chirp generator's running state.
3. **Complex multiplier:** tx_real = (S_R·csk_out_r) – (S_I·csk_out_i), tx_imag = (S_R·csk_out_i) + (S_I·csk_out_r) — the standard (a+jb)(c+jd) expansion, registered (one clock of latency), implementing the CSS modulation multiply (S_n × chirp_sample) established throughout this project.
4. **tx_done chain:** ever_run arms once csk_running first asserts (preventing a false trigger at reset), tx_done_level goes high the cycle csk_running drops while armed, posedge_detector converts that level into a single-cycle pulse (tx_done_raw), and **two** additional flip-flop stages (tx_done_dly1→tx_done_dly2) delay the final tx_done output by 2 total cycles beyond the raw pulse — matching the previously-implemented fix ensuring tx_done asserts strictly after the last transmitted sample, not concurrently with it, with the extra cycle you specifically requested added earlier in this project's tx_done timing correction work.

Mode Dependence: None — this integration layer is rate-independent (the CSK Generator, DQPSK encoder, and multiplier are all rate-agnostic per earlier module summaries); rate-dependent behavior lives entirely upstream in e_o_demux/symbol_mapper/interleaver/form_ppdu.

12 - Controller (controller.v)

Function: Top-level transmit sequencing block. Tracks whether a frame transmission is in progress and, from that single status bit, derives zero_pad_en to arm the zero-padding stage; in hybrid-rate builds, additionally derives the mode signal that selects between 1 Mb/s and 250 kb/s for every downstream block (SHR generator, symbol mapper, e/o demux, zero-padding).



Key Signals:

- Inputs: start_tx (frame-start request), tx_end (frame-complete indication), clk, rst_n, plus payload_length, fifo_empty, demux_valid, interleaver_valid (present on the port list but not used in the transition logic)
- Outputs: mode (rate select, hybrid builds only), zero_pad_en (enables the zero-padding datapath)

Behavior:

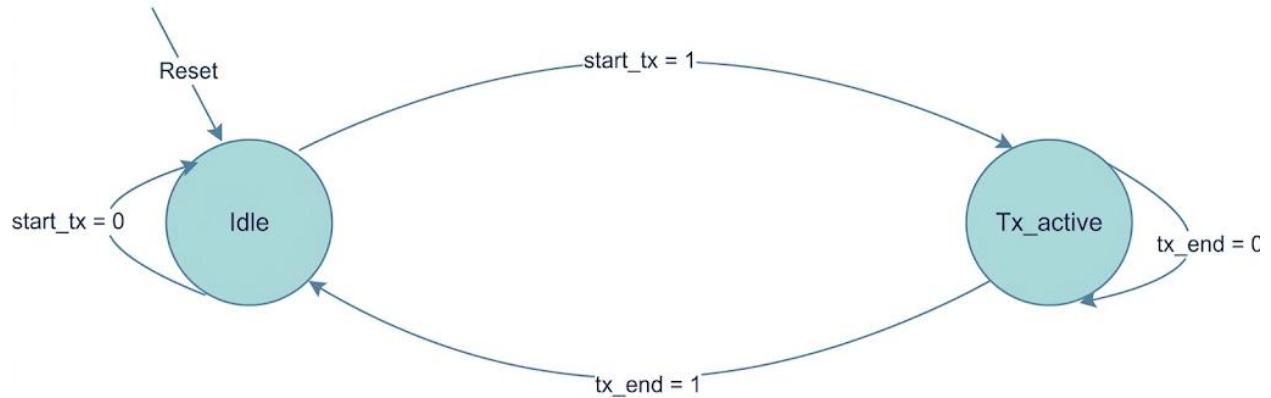
- Implemented as a 2-state busy/idle flag (idle_flag) rather than an explicit case-based state register — functionally a Moore machine with Idle (idle_flag = 1, zero_pad_en = 0) and Tx_active (idle_flag = 0, zero_pad_en = 1) states, but coded as a single bit set/cleared directly by start_tx/tx_end rather than through named states.
- tx_end is evaluated ahead of start_tx in the same always block, so if both were asserted in the same cycle the FSM ends the current frame rather than re-triggering a back-to-back transmission; there is no dedicated handling for start_tx being re-asserted before tx_end completes.
- In hybrid mode (rate_mode = "H"), a side counter (counter_for_mode_detection) increments once per start_tx pulse in parallel with the same busy/idle flag, and mode is derived combinatorially from that count — this rides on top of the 2-state machine rather than adding separate states, and depends on start_tx being held for a specific number of cycles (1 vs. 2) by the driving logic upstream, a timing contract the controller itself does not enforce or verify.

Mode Dependence: Rate-mode dependent only in the assign mode expression and its associated counter logic (present solely in the "H" generate branch); the underlying busy/idle sequencing (idle_flag, zero_pad_en) is identical across "F", "S", and "H" builds.

2. FSM Diagrams

This section documents every finite state machine in the design: its states, transition conditions, and the timing behavior it implements.

2.1 Transmit Control FSM



Controls: Gates the transmit datapath; derives `zero_pad_en` and (in hybrid mode) `mode` from a single busy/idle flag.

States

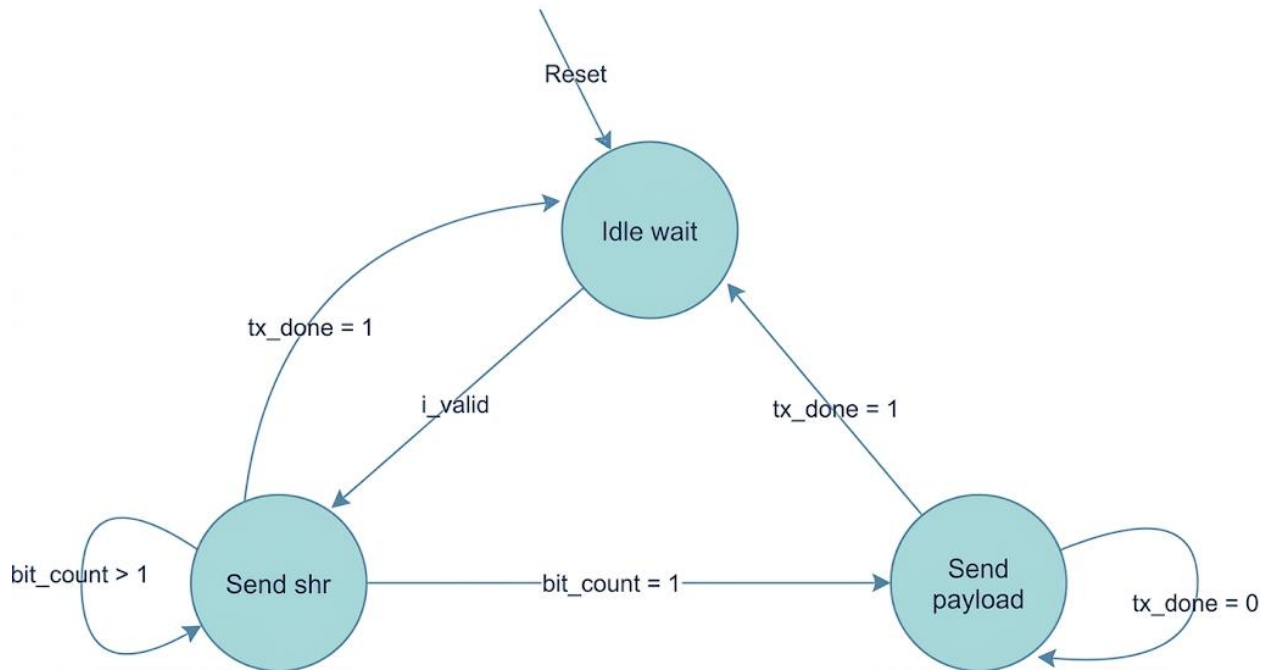
- **Idle** — no transmission; `zero_pad_en = 0`
- **Tx_active** — transmitting; `zero_pad_en = 1`

Transitions

- Idle → Tx_active: `start_tx = 1`
- Tx_active → Idle: `tx_end = 1`
- Self-loops otherwise (`start_tx = 0` / `tx_end = 0`)
- `tx_end` checked before `start_tx`, so it wins if both assert same cycle

Reset: Synchronous active-low `rst_n` forces straight to Idle, same path as `tx_end`.

2.2 Frame Formation FSM



Controls: Assembles the outgoing PPDU by sequencing preamble+SFD (SHR) chips followed by the delayed payload chips onto o_i/o_q/o_valid.

States

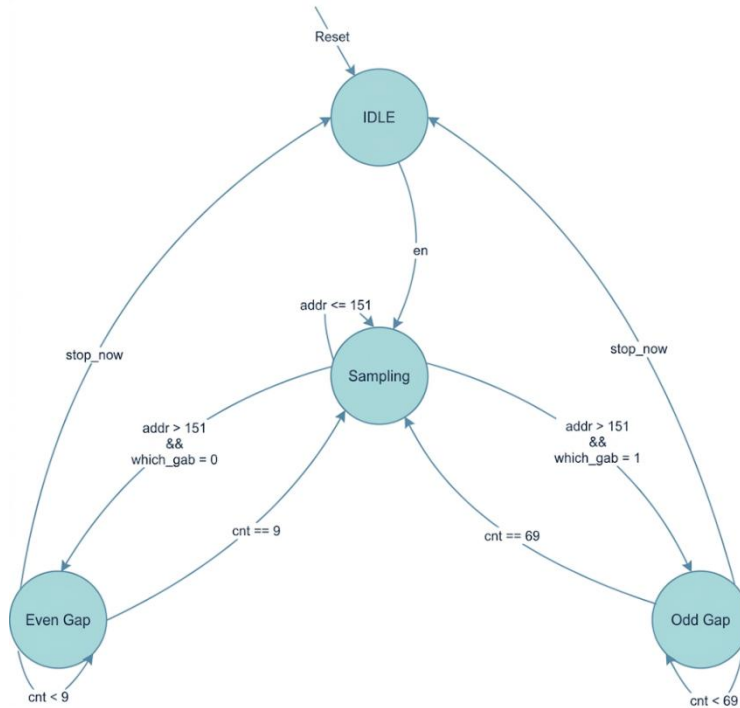
- **Idle_wait** — waiting for the first chip of a frame; outputs held at 0
- **Send_shr** — streaming preamble+SFD chips out
- **Send_payload** — streaming the delayed payload chips out (via the internal delay line)

Transitions

- Idle_wait → Send_shr: i_valid = 1 (first preamble chip loads)
- Send_shr → Send_payload: bit_count = 1 (last SHR chip about to go out)
- Send_shr self-loop: bit_count > 1 (still streaming SHR)
- Send_payload self-loop: tx_done = 0 (still streaming payload)
- Send_shr → Idle_wait: tx_done = 1
- Send_payload → Idle_wait: tx_done = 1

Reset: Synchronous active-low rst_n forces state to Idle_wait, clears all output/data/count registers.

2.3 CSK Chip Generator FSM



Controls: Streams the CSK ROM chip sequence into `csk_out_r/csk_out_i`, inserting a fixed-length gap between repetitions and alternating gap length each cycle through.

States

- **Idle** — not running; outputs held at 0, waiting for `en`
- **Sampling** — `addr_ptr ≤ 151`; streaming ROM chips out
- **Even_gap** — `which_gab = 0`; outputting 0 for 10 cycles
- **Odd_gap** — `which_gab = 1`; outputting 0 for 70 cycles

Transitions

- Idle → Sampling: `en = 1`
- Sampling self-loop: `addr_ptr ≤ 151` (still fetching)
- Sampling → Even_gap: `addr_ptr > 151 && which_gab = 0`
- Sampling → Odd_gap: `addr_ptr > 151 && which_gab = 1`
- Even_gap → Sampling: `cnt = 9` (also resets `addr_ptr`, toggles `which_gab` → 1)
- Odd_gap → Sampling: `cnt = 69` (also resets `addr_ptr`, toggles `which_gab` → 0)
- Even_gap self-loop: `cnt < 9`
- Odd_gap self-loop: `cnt < 69`
- Even_gap → Idle: `stop_now`
- Odd_gap → Idle: `stop_now`

Reset: Synchronous active-low `rst_n` forces `running = 0` (Idle), zeroes `addr_ptr`, `sambl_cnt`, `which_gab`, outputs.

3. Word Length / ROM Design Decisions & MSE Justification

This section documents every fixed-point format decision made in the design, the reasoning behind each word-length and ROM sizing choice, and the quantitative (MSE) evidence supporting it.

3.1 Fixed-Point Format Summary

Signal / ROM	Format (WL, IB, FL) & Range
Chirp/CSK ROM	Q1.4, WL=6: 1 sign + 1 int + 4 frac, range [-2, 1.9375]
QPSK/DQPSK symbol	Q1.0, WL=2: 1 sign + 1 int + 0 frac, range [-2,1]
Multiplier / TX output	[Q3.4, WL=8: 1 sign + 3 int + 4 frac, range [-8,7.9375]

3.2 Word-Length Selection Rationale

Chirp / CSK ROM — Q1.4, WL = 6 (1 sign + 1 int + 4 frac)

Integer bits

- **Peak signal magnitude:** the baseband chirp/CSK waveform consists of normalized sinusoids ($\cos\theta$ and $\sin\theta$) whose maximum dynamic amplitude strictly falls within the range $[-1.0, +1.0]$.
- **Integer bit justification:** a single integer bit, together with 1 sign bit, provides a two's-complement dynamic range of $[-2.0, +1.9375]$. This safely accommodates the peak values of ± 1.0 without overflow or dynamic clipping.

Fraction bits

- **Quantization noise and resolution:** 4 fraction bits provide a step resolution of $2^{-4} = 0.0625$.
- **Precision trade-off:** this resolution keeps the quantization noise floor negligible relative to the modulation threshold, preserving chirp correlation properties while minimizing LUT/ROM memory usage.

QPSK / DQPSK Symbol — Q1.0, WL = 2 (1 sign + 1 int + 0 frac)

Integer bits

- **Peak signal magnitude:** standard constellation mapping for BPSK/QPSK/DQPSK symbols uses normalized unit values (± 1).
- **Integer bit justification:** 1 integer bit combined with 1 sign bit exactly represents the discrete constellation points $\{-1, +1\}$ within the range $[-2, +1]$, completely avoiding overflow during symbol generation.

Fraction bits

- **Discrete mapping:** constellation values are strictly integral points (± 1). Zero fraction bits are required since there are no intermediate fractional values in the ideal mapping stage, minimizing routing overhead and register width.

Multiplier / TX Output — Q3.4, WL = 8 (1 sign + 3 int + 4 frac)

Integer bits

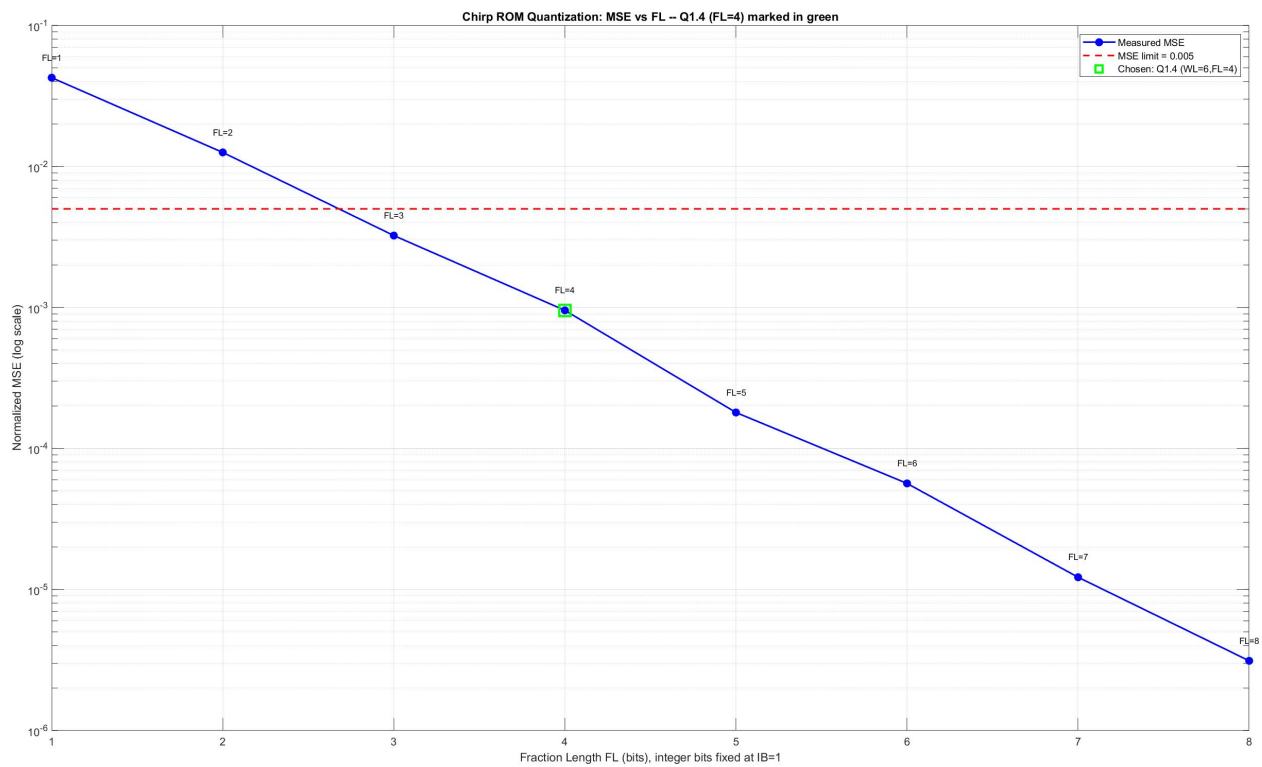
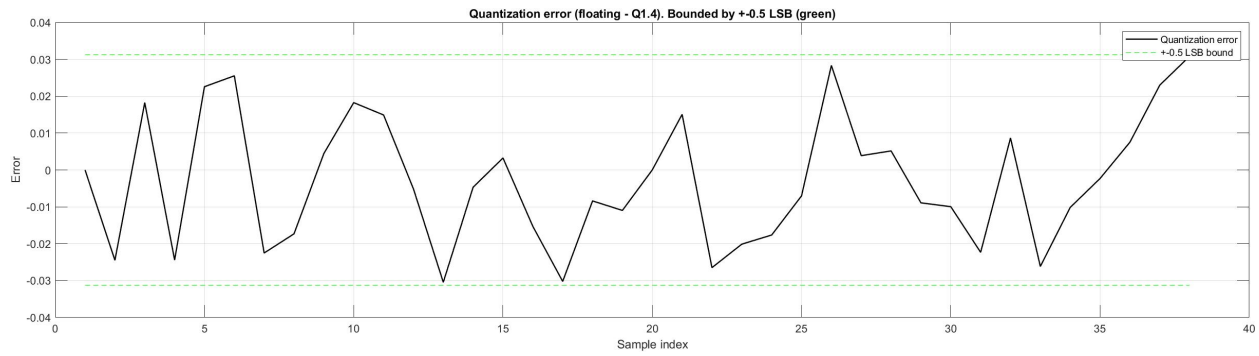
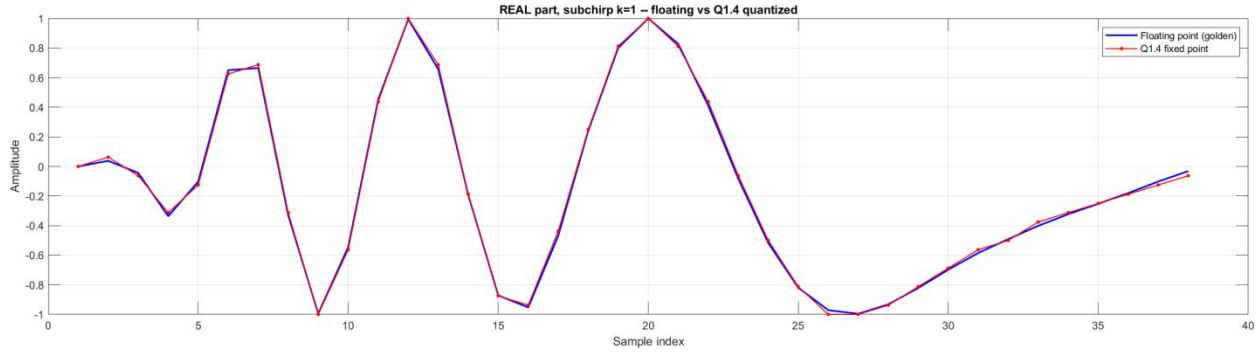
- **Intermediate operations and guard bits:** the output is formed by multiplying the chirp reference signal (Q1.4) with the constellation phase/symbol (Q1.0), followed by digital sum/difference operations such as complex baseband mixing or pulse-shaping accumulation.
- **Maximum dynamic growth:** a direct multiplication of maximum magnitudes yields $1.0 \times 1.0 = 1.0$. However, intermediate summation stages or filter taps can cause bit growth and dynamic overshoot up to roughly ± 3.0 to ± 4.0 .
- **Integer bit justification:** 3 integer bits, plus 1 sign bit, expand the range to $[-8.0, +7.9375]$, providing 2 additional guard bits to prevent overflow or clipping during peak vector additions.

Fraction bits

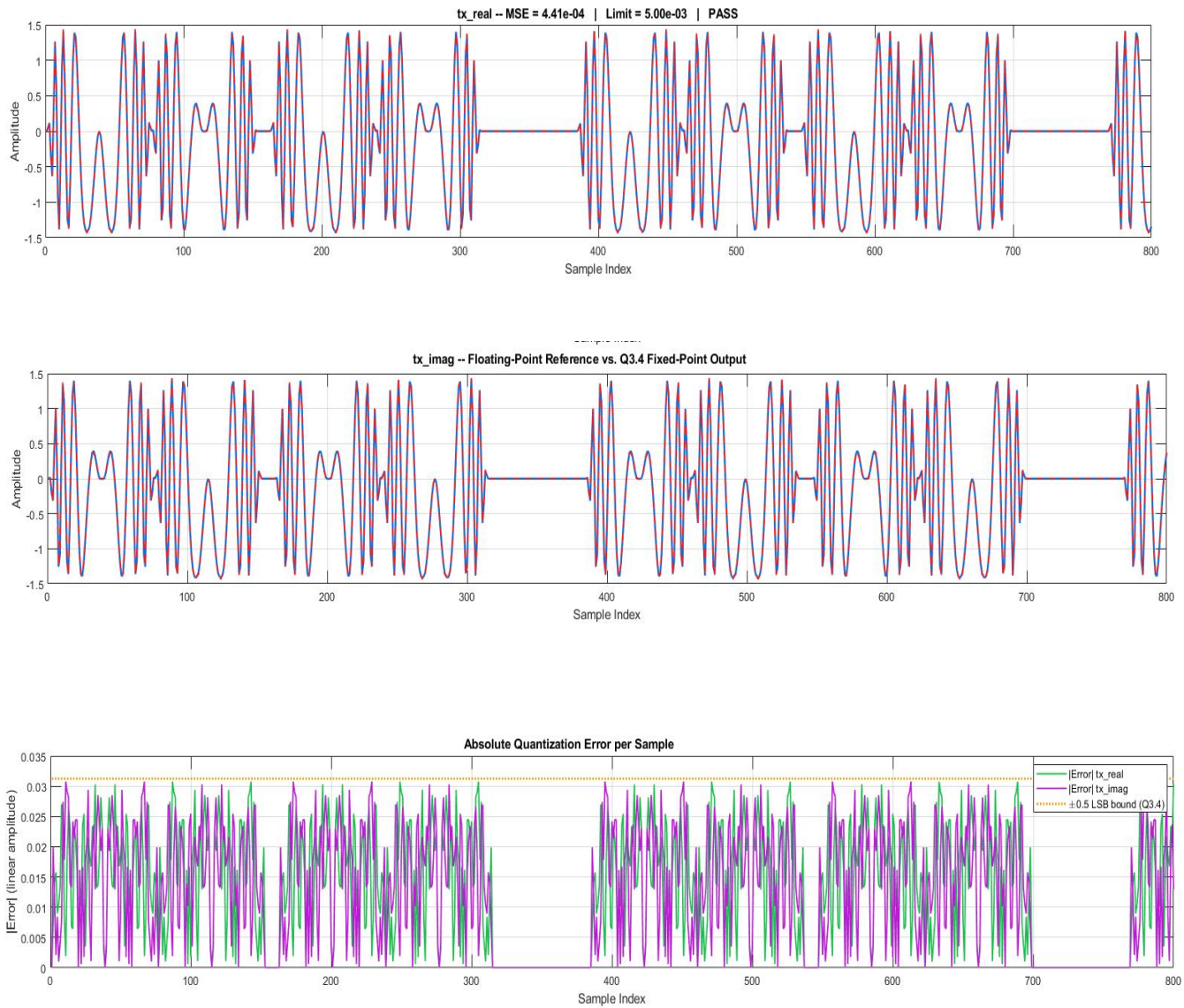
- **Precision retention:** 4 fraction bits ($2^{-4} = 0.0625$) are retained to preserve the phase and amplitude fidelity of the incoming chirp signal without accumulating truncation/rounding error at the DAC/TX interface.

3.3 MSE Sweep & Justification

1 - CSK ROM word-length sweep — floating-point vs. Q1.4 quantized output, quantization error, and MSE vs. fraction length



2 - Multiplier / TX output word-length validation — floating-point vs. Q3.4 fixed-point (real/imag), and absolute quantization error per sample



3.4 ROM Sizing & Addressing

Chirp/CSK ROM

- **Depth:** 152 entries (address range 0–151), covering one full chirp/CSK chip sequence per symbol period.
- **Word width:** 6 bits (Q1.4 fixed-point: 1 sign + 1 integer + 4 fraction), matching the format justified in Section 3.2.
- **Addressing scheme:** A single 8-bit address counter (addr_ptr) increments by one every clock cycle while streaming ($\text{addr_ptr} \leq 151$), sequentially walking the ROM from index 0 to 151. Once addr_ptr exceeds 151, the block exits the ROM-read region and enters the gap period (10 or 70 cycles, alternating even/odd), after which addr_ptr resets to 0 and the sequence repeats. No random or computed addressing is used — access is purely sequential.
- **Content generation and verification:** ROM contents were generated from, and verified against, the MATLAB floating-point golden model. The fixed-point (Q1.4) ROM output was compared sample-by-sample against the floating-point reference, with the resulting quantization error bounded within ± 0.5 LSB across the full sequence. A fraction-length (FL) sweep from FL=1 to FL=6 was run to select the word length, with normalized MSE measured at each candidate; FL=4 (Q1.4, WL=6) was chosen as it is the smallest word length whose MSE falls below the 0.005 limit, giving the best trade-off between numerical accuracy and ROM storage/LUT cost. This sweep and the resulting pass/fail against the MSE limit are documented in Section 3.3.

```

=====
Q1.4 CHIRP ROM: MSE SWEEP vs FRACTION LENGTH
=====

Golden floating-point chirp sequence generated.
  Size : 38 samples/subchirp x 4 subchirps (chirpIndex=1)

--- STEP 1: Integer-bit budget check ---
Max |real| = 1.000000, Max |imag| = 0.996582
IB=1 max representable magnitude = 2.0000 -- assumption holds.

FL=1 WL=3 (Q1.1) | MSE = 0.04257635 [FAIL]
FL=2 WL=4 (Q1.2) | MSE = 0.01256907 [FAIL]
FL=3 WL=5 (Q1.3) | MSE = 0.00323462 [PASS]
FL=4 WL=6 (Q1.4) | MSE = 0.00095340 [PASS]
FL=5 WL=7 (Q1.5) | MSE = 0.00017993 [PASS]
FL=6 WL=8 (Q1.6) | MSE = 0.00005650 [PASS]
FL=7 WL=9 (Q1.7) | MSE = 0.00001221 [PASS]
FL=8 WL=10 (Q1.8) | MSE = 0.00000312 [PASS]

--- STEP 3: Target format check -- Q1.4 (WL=6, FL=4) ---
MSE at FL=4 : 0.00095340
RESULT: Q1.4 (WL=6) PASSES the 0.005 MSE requirement.
Margin: 5.24x below the limit.

=====
SWEEP COMPLETE. Q1.4 (WL=6) is CONFIRMED PASSING per the 0.005 MSE criterion.
=====
fx >> |

```

4. Verification Methodology & Results

This section describes how correctness was established, what was compared against what, and summarizes the results.

4.1 Verification Strategy

The verification methodology for the transmit chain is built around continuous, automated comparison against a golden reference model, rather than manual waveform inspection or isolated directed tests. This section describes the approach used to validate correctness across the full design.

Golden-Model Comparison

Each block in the transmit chain is validated against a reference model implemented independently in floating-point/fixed-point MATLAB. For a representative set of payload lengths — chosen to exercise both minimal and maximal frame sizes, including zero-length and zero-padding-heavy edge cases — the reference model's expected outputs are pre-computed and loaded into the simulation environment. During simulation, every sample produced by the RTL is compared directly against the corresponding golden sample, cycle by cycle, for the full duration of the transmission.

This approach ensures that correctness is verified against an independently derived expected behavior, rather than against assumptions embedded in the RTL itself, giving confidence that the implementation matches the intended algorithm rather than simply being internally self-consistent.

Self-Checking Testbenches

The comparison logic is fully automated within the testbench, requiring no manual review to determine pass or fail. For every test case, each pipeline stage maintains independent counters tracking the number of matching and mismatching samples observed. At the conclusion of each test, a summary report is generated showing the error and match counts for every stage, along with a single overall verdict: the test case is marked as passed only if every stage reports zero errors, and failed otherwise.

Multiple payload-length scenarios are executed sequentially within a single simulation run, allowing the full range of supported frame sizes to be verified without manual reconfiguration between runs. Where the design supports multiple data rates within a single build (hybrid mode), the stimulus is additionally varied to exercise each supported rate through the same instantiated design, confirming that rate selection and the corresponding datapath behavior are both verified under the same test infrastructure.

Stages and Signals Verified

Verification is not limited to the final transmitted output. Internal signals are observed at each significant stage of the datapath, providing fine-grained fault isolation rather than a single pass/fail judgment on the end-to-end result. The stages independently checked include:

- Zero-padding output
- Even/odd data demultiplexing (including validity signaling)
- Symbol mapping output
- Interleaver output (including per-stream validity signaling)
- Phase mapping (QPSK) output
- Differential encoding (DQPSK) output
- Final transmitted in-phase and quadrature samples

By checking each stage independently rather than only the final output, a mismatch can be immediately attributed to the specific stage where the RTL first diverges from the reference model, rather than requiring manual backtracking through the pipeline to localize a fault.

Mismatch Logging and Traceability

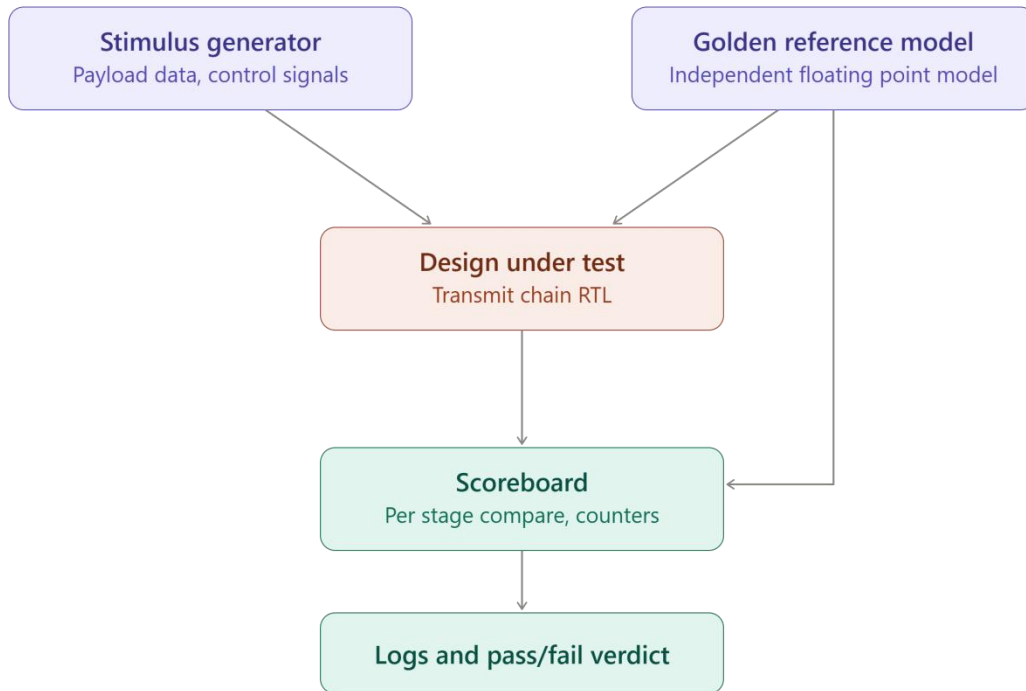
Every sample comparison — whether matching or mismatching — is recorded to a dedicated log per pipeline stage, including the simulation time, sample index, and both the RTL-observed and reference values. This produces a complete, timestamped audit trail for each stage of verification, allowing any discrepancy to be traced back to the exact simulation cycle and sample index at which it occurred, without needing to re-run or re-instrument the simulation to reproduce the failure.

Simulation Environment

Simulations are executed in an industry-standard HDL simulator, with automated scripts handling library setup, RTL and testbench compilation, and simulation invocation for each supported data-rate configuration. Fixed-point display formats are configured to match the word-length and fraction-length conventions established for each signal (Section 3.1), enabling direct visual inspection of quantized values during debug when needed, in addition to the automated pass/fail scoreboard.

Captured transmit-chain output samples are additionally exported for offline post-processing, independent of the pass/fail checking infrastructure. This supports downstream analyses — such as the mean-squared-error sweep used to justify fixed-point word-length selection (Section 3.3) — using the same captured data that was validated during functional verification.

4.2 Testbench Architecture



Stimulus generation. Test stimulus is applied by driving the DUT's input ports directly (clock, reset, payload data, payload length, and the start-of-transmission control signal). Payload content is supplied from pre-defined data sets covering a representative range of payload lengths — including boundary cases such as an empty payload and the maximum supported length — so that both nominal and edge-case framing behavior are exercised. Where the design supports more than one data rate, the timing of the start-of-transmission signal is varied to force each supported rate through the same instantiated design.

Golden reference source. Expected outputs are not derived from the RTL itself, but from an independently implemented floating-point/fixed-point reference model. This reference model computes the expected value at each stage of the transmit chain for the same input stimulus, and its outputs are loaded into the testbench as the "golden" data set against which the RTL is judged.

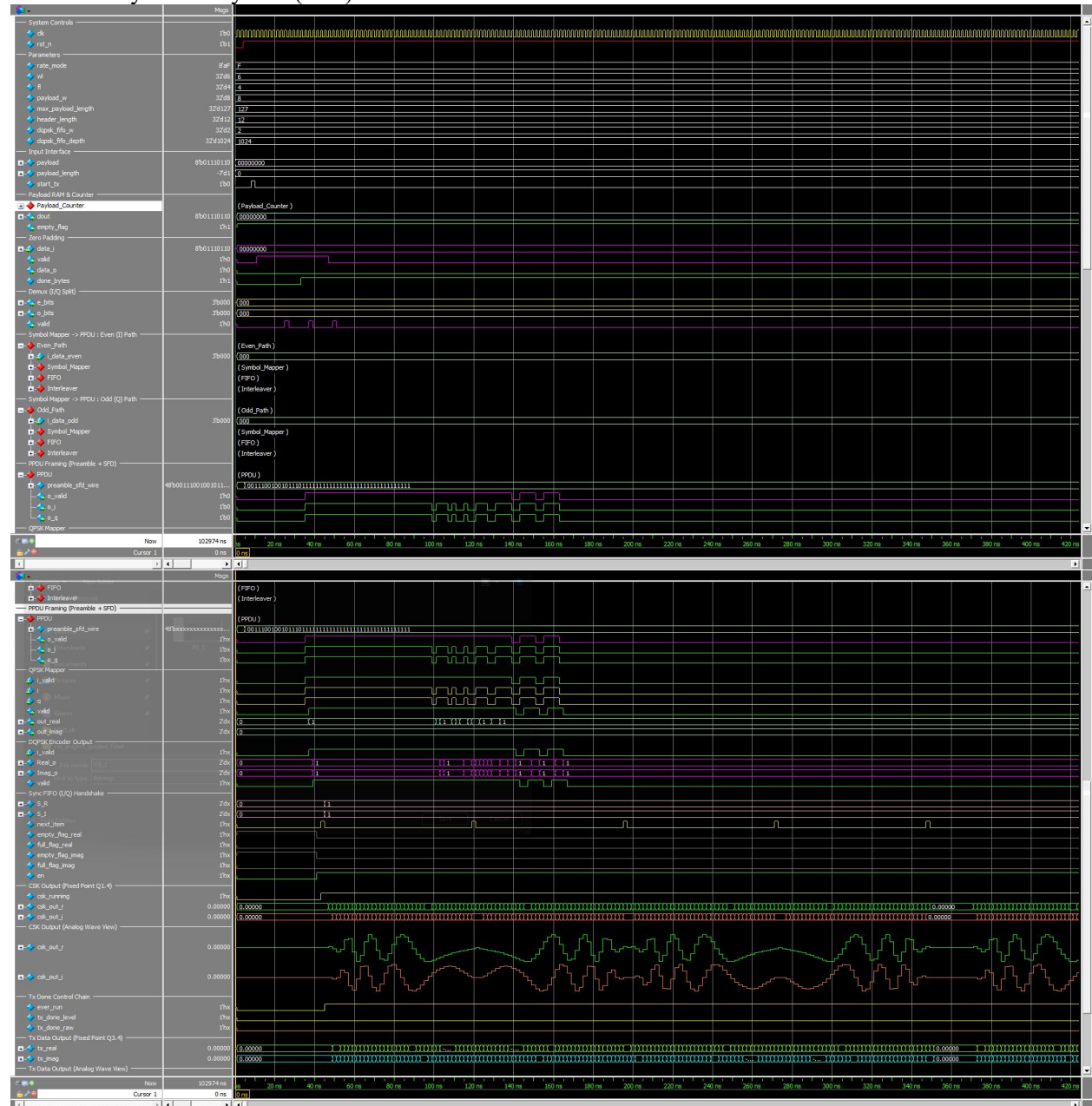
Pass/fail comparison mechanism. Internal DUT signals are tapped at each major processing stage — not only the final transmitted output — and compared sample-by-sample, cycle-by-cycle, against the corresponding golden reference values. Each stage keeps independent match/mismatch counters. At the end of each test, these are combined into a single verdict: the test passes only if every stage recorded zero mismatches; any nonzero error count anywhere in the chain fails the test. Every comparison, matching or not, is timestamped and logged per stage, allowing any failure to be traced back to the exact cycle, sample, and stage at which the RTL first diverged from the reference model.

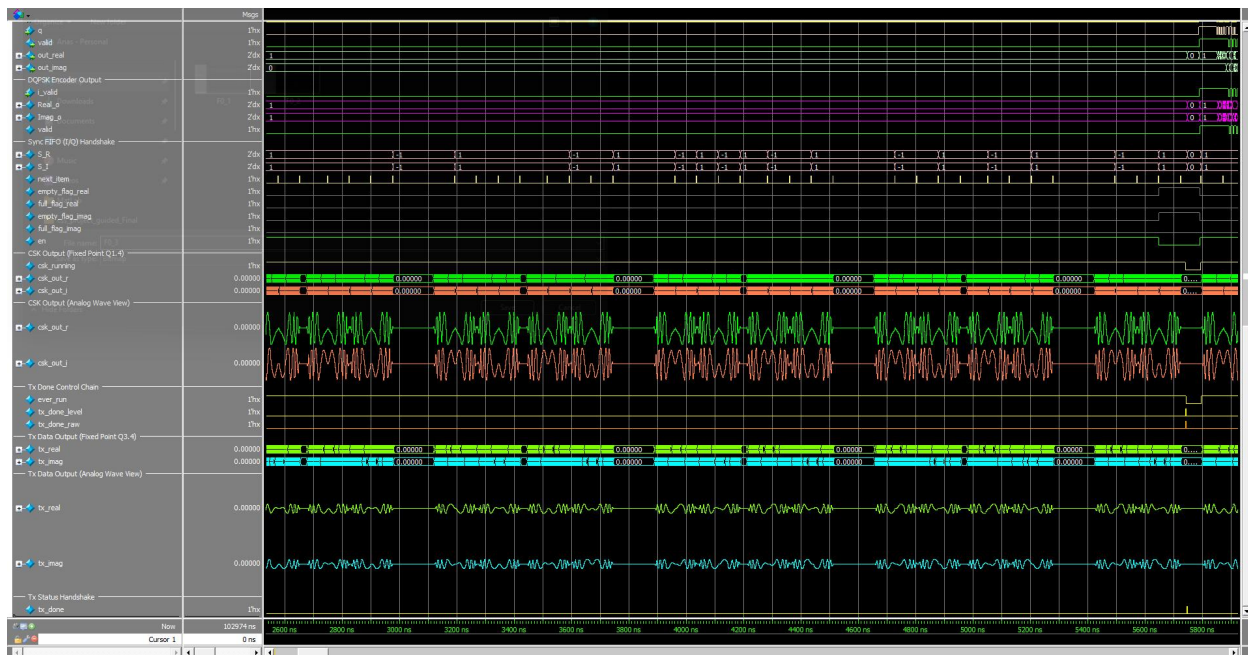
4.3 Test Cases & Coverage

Test Case	Description / Coverage Goal
Case 1	Testing Fast Mode (1Mbps) for four different payloads [0, 1, 28, 127] Bytes
Case 2	Testing Slow Mode (250Kbps) for four different payloads [0, 1, 28, 127] Bytes
Case 3	Testing Hybrid Mode (1Mbps / 250Mbps) for four different payloads [0, 1, 28, 127] Bytes

A - Case 1, Testing Fast Mode (1Mbps) :

Test 1 - 0 Bytes of Payload (Fast) :





Reset is OK.

rst deasserted, starting test case 0

Rate Mode = F

Payload Length = 0

=====

5748 ns: tx_done asserted.

TX completed. Captured 2869 samples.

Test Case 0 Results

Payload Length = 0

QPSK errors = 0

QPSK Correct = 60

Interleaver errors = 0

Interleaver Correct = 12

Symbol Mapper errors = 0

Symbol Mapper Correct = 3

Zero Padding errors = 0

Zero Padding Correct = 18

Demux errors = 0

Demux Correct = 3

REAL errors = 0

IMAG errors = 0

REAL Correct = 2869

IMAG Correct = 2869

DQPSK errors = 0

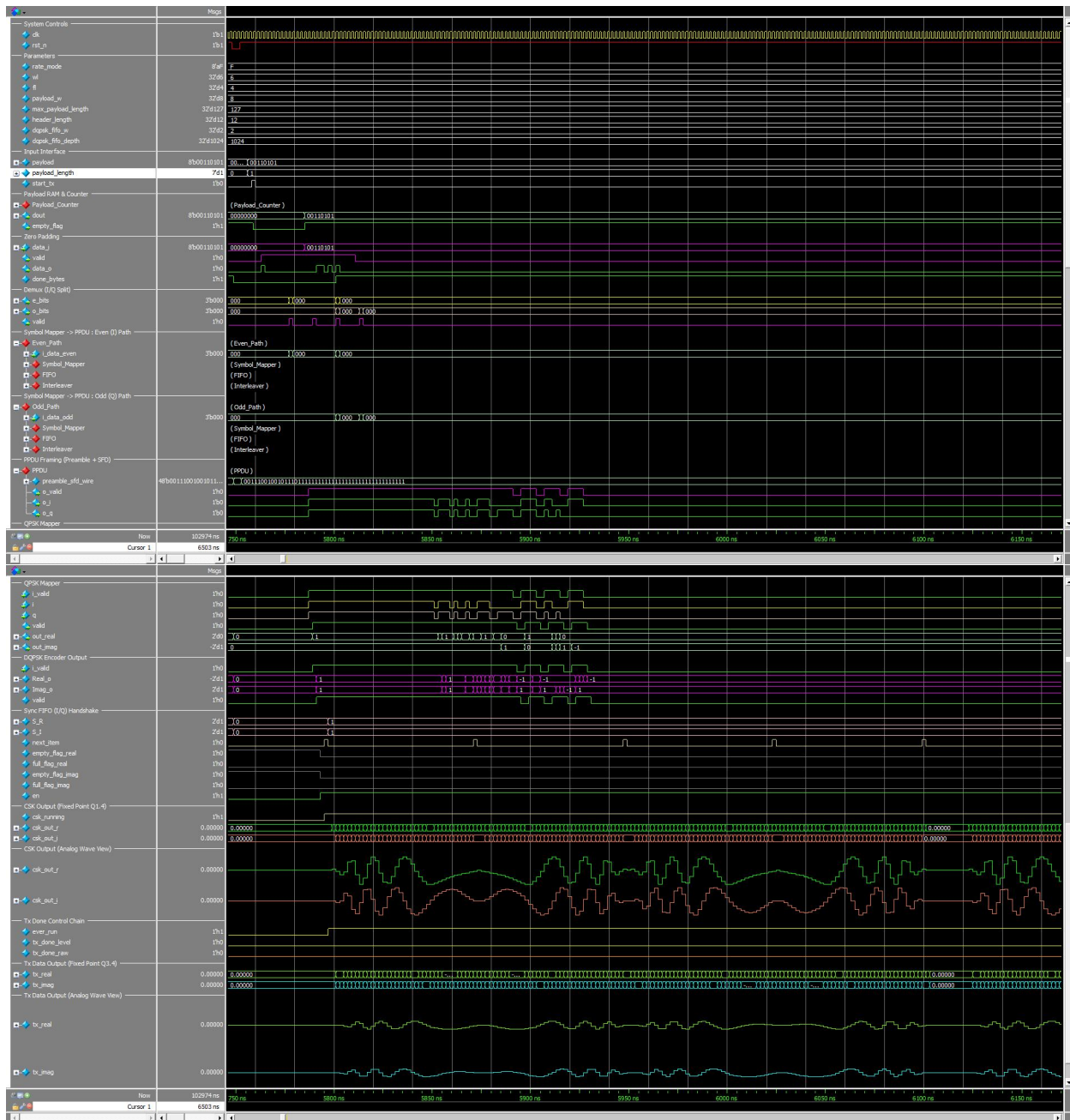
DQPSK Correct = 60

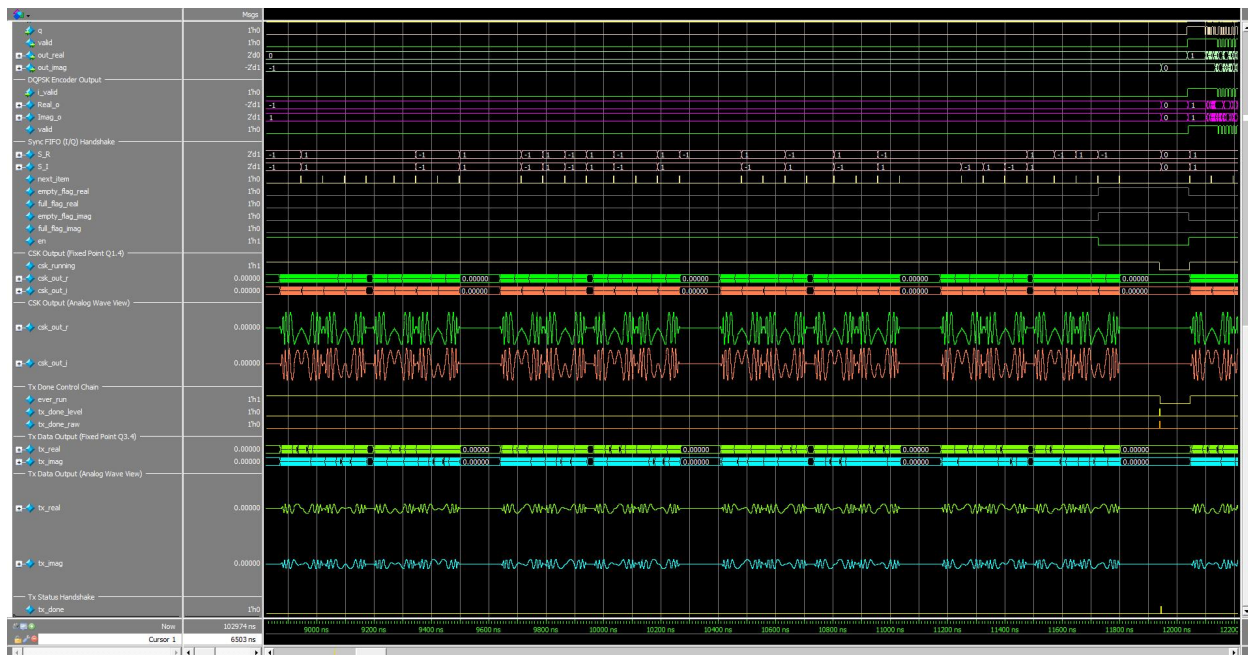
Total captured samples = 2869

***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****

test cases no = 0 completed.

Test 2 - 1 Bytes of Payload (Fast) :





Reset is OK.

rst deasserted, starting test case 1

Rate Mode = F

Payload Length = 1

=====

11944 ns: tx_done asserted.

TX completed. Captured 3092 samples.

Test Case 1 Results

Payload Length = 1

QPSK errors = 0

QPSK Correct = 64

Interleaver errors = 0

Interleaver Correct = 16

Symbol Mapper errors = 0

Symbol Mapper Correct = 4

Zero Padding errors = 0

Zero Padding Correct = 24

Demux errors = 0

Demux Correct = 4

REAL errors = 0

IMAG errors = 0

REAL Correct = 3092

IMAG Correct = 3092

DQPSK errors = 0

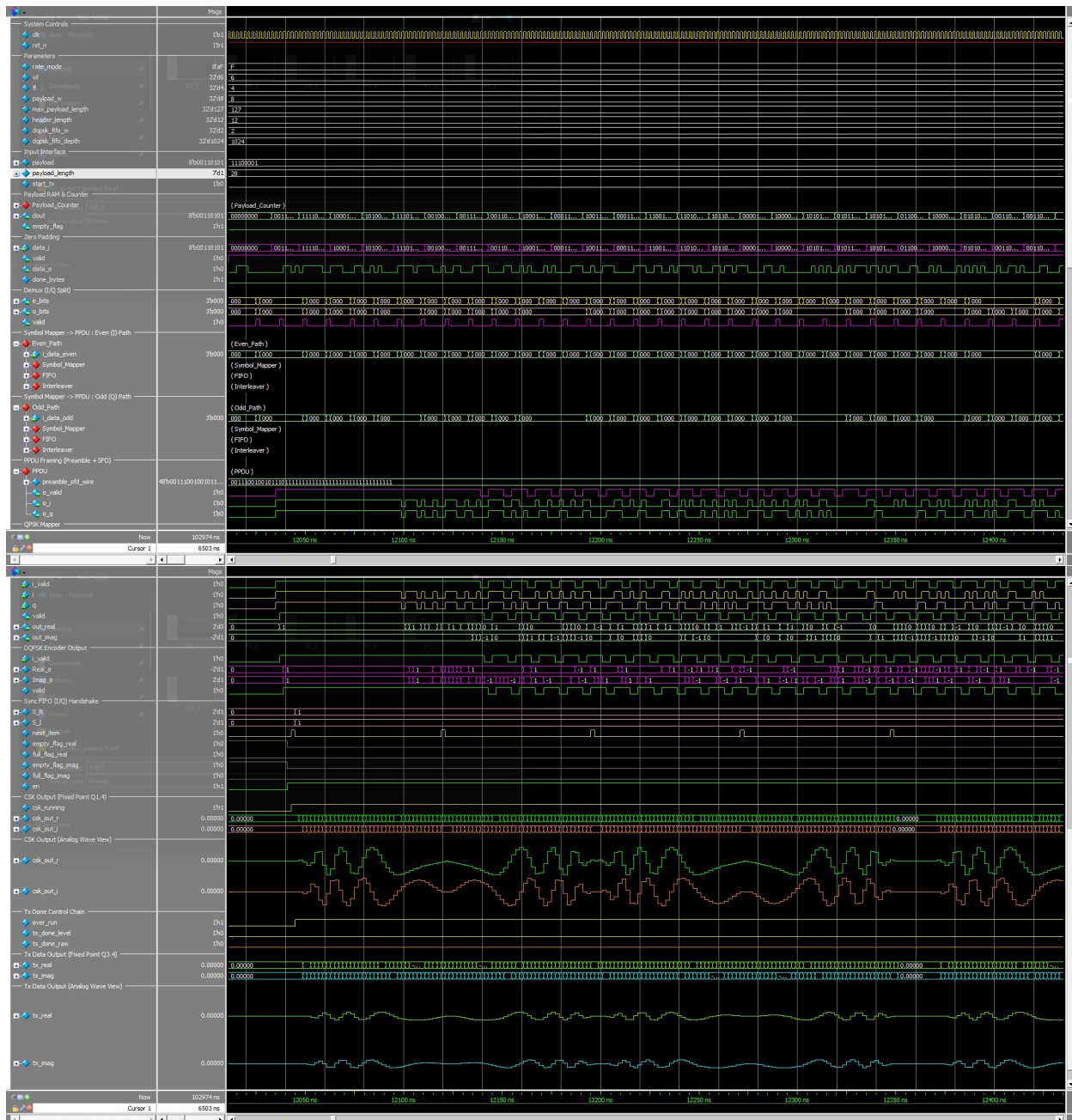
DQPSK Correct = 64

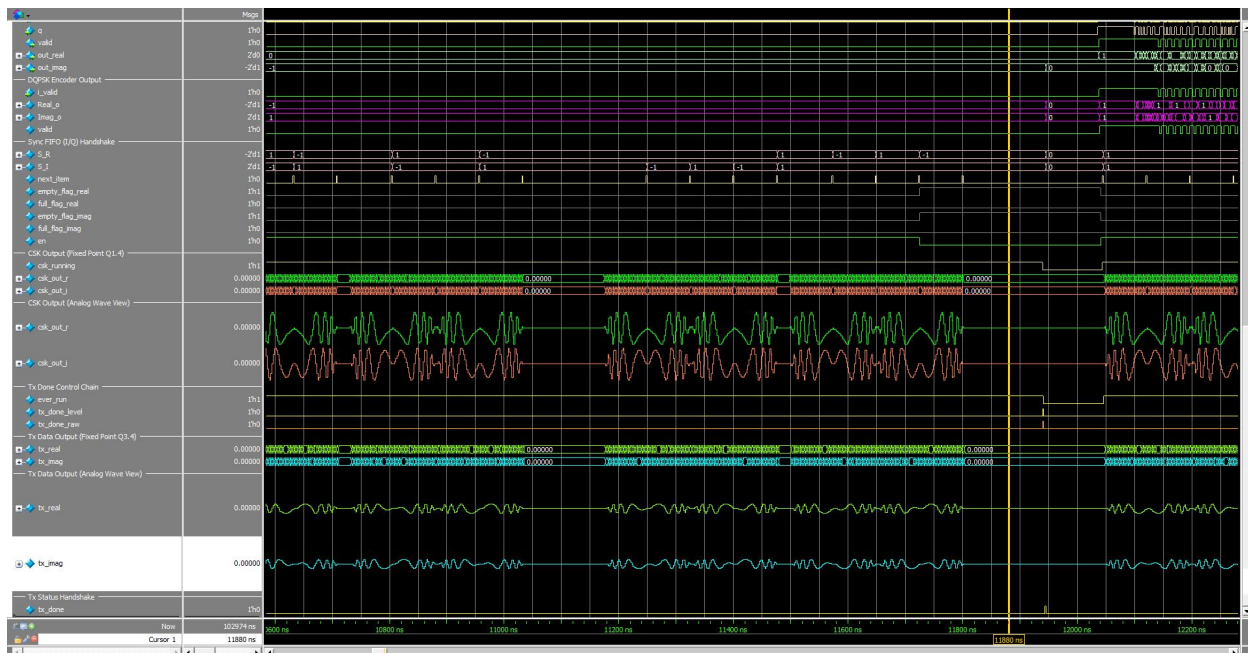
Total captured samples = 3092

***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****

test cases no = 1 completed.

Test 3 - 28 Bytes of Payload (Fast):





```

Reset is OK.
rst deasserted, starting test case 2
Rate Mode      = F
Payload Length  = 28
=====
32016 ns: tx_done asserted.
TX completed. Captured 10003 samples.

```

```

-----
Test Case 2 Results
Payload Length = 28
QPSK errors = 0
QPSK Correct = 208
Interleaver errors = 0
Interleaver Correct = 160
Symbol Mapper errors = 0
Symbol Mapper Correct = 40
Zero Padding errors = 0
Zero Padding Correct = 240
Demux errors = 0
Demux Correct = 40
REAL errors = 0
IMAG errors = 0
REAL Correct = 10003
IMAG Correct = 10003
DQPSK errors = 0
DQPSK Correct = 208
Total captured samples = 10003

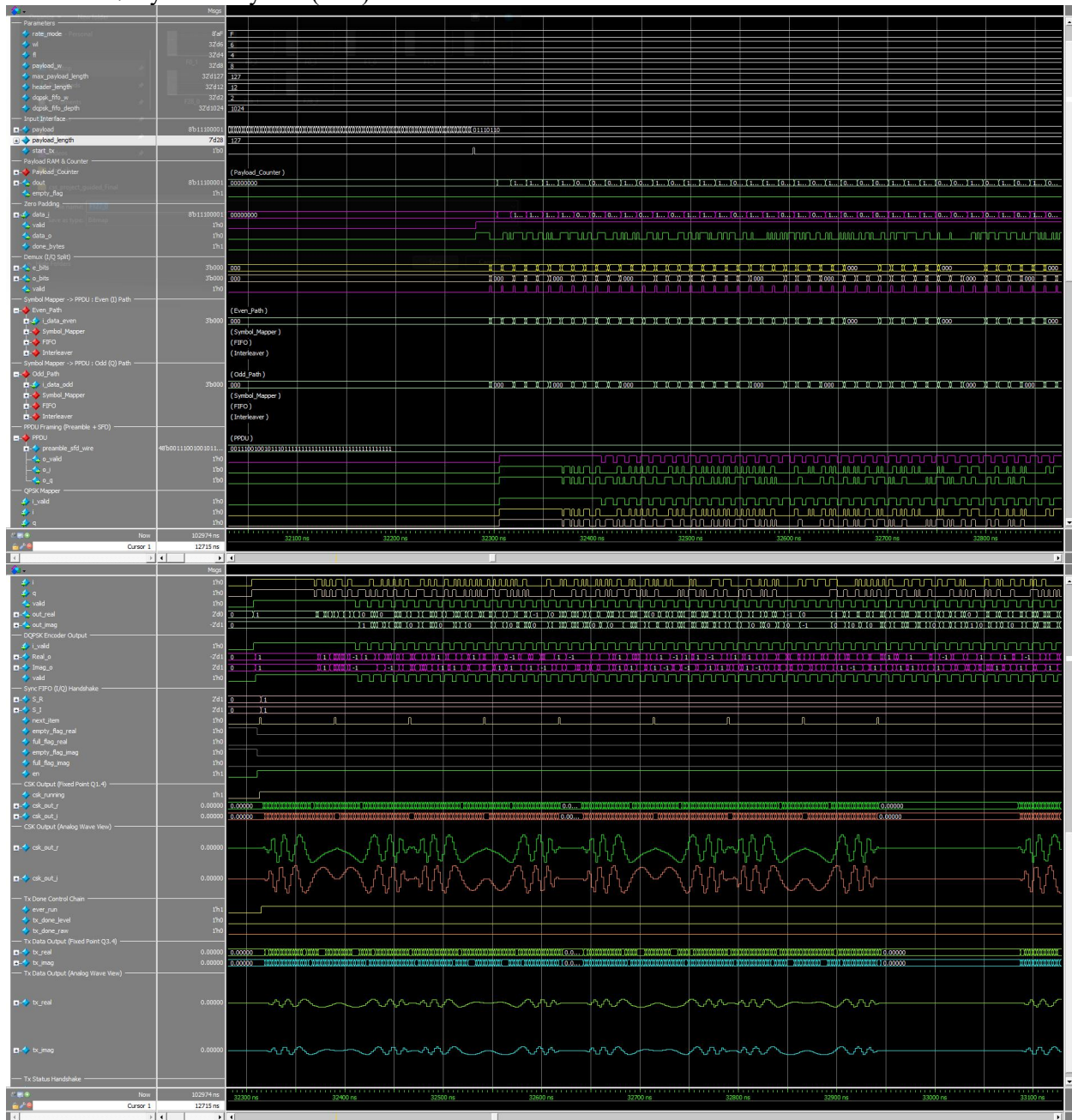
```

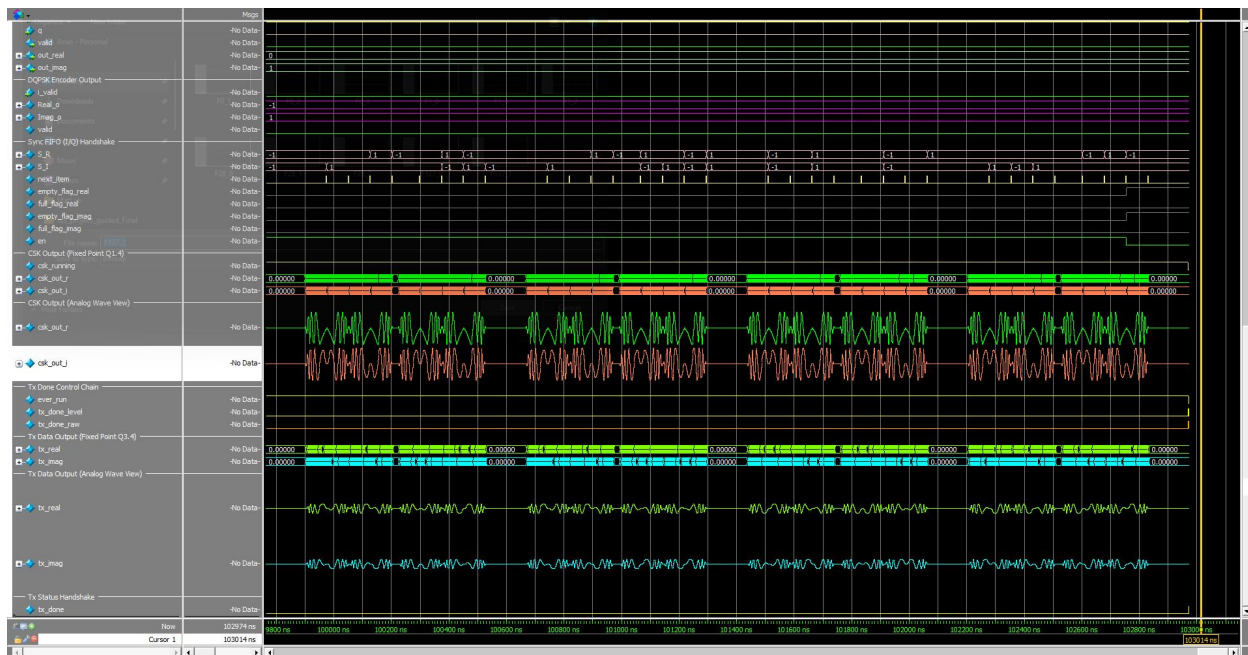
```

-----
***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
test cases no = 2 completed.

```

Test 4 - 127 Bytes of Payload (Fast) :





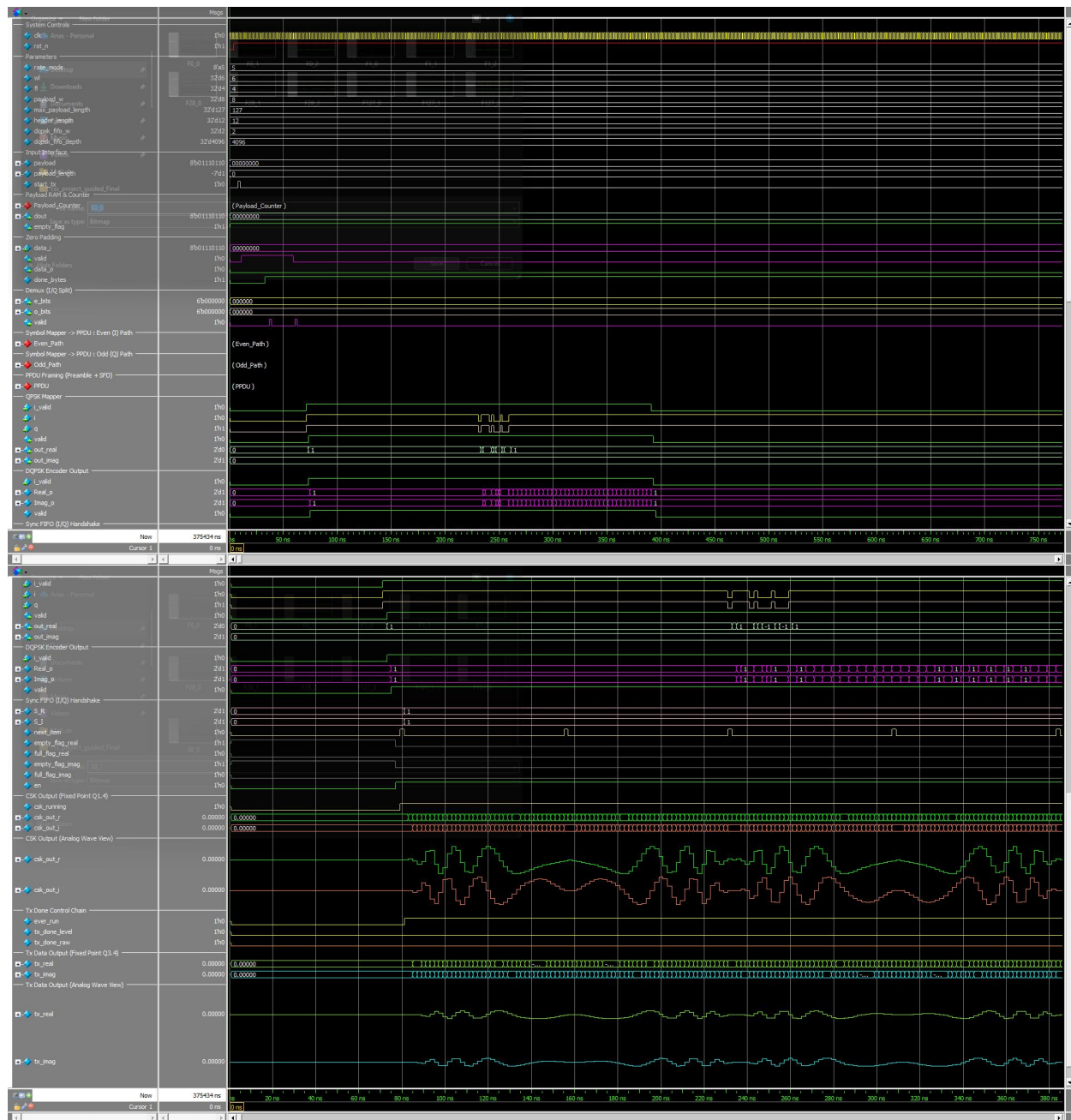
```

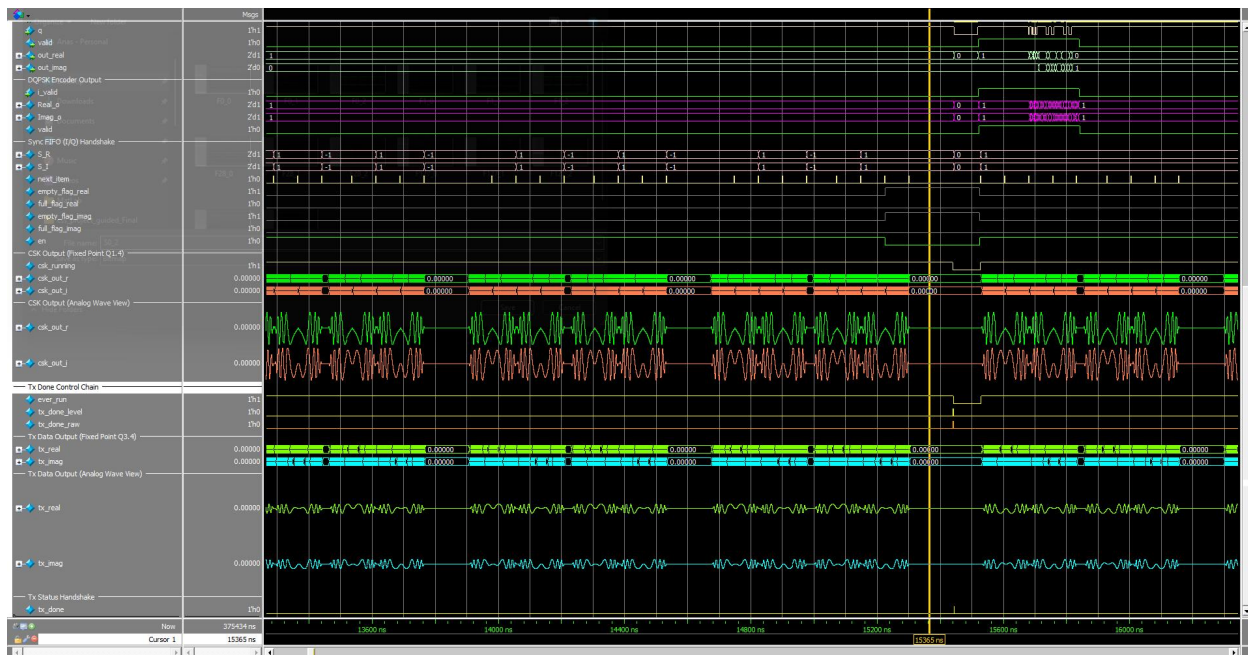
# Reset is OK.
# rst deasserted, starting test case 3
# Rate Mode      = F
# Payload Length  = 127
# =====
#           102974 ns: tx_done asserted.
# TX completed. Captured 35347 samples.
# -----
# Test Case 3 Results
# Payload Length = 127
# QPSK errors = 0
# QPSK Correct = 736
# Interleaver errors = 0
# Interleaver Correct = 688
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 172
# Zero Padding errors = 0
# Zero Padding Correct = 1032
# Demux errors = 0
# Demux Correct = 172
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 35347
# IMAG Correct = 35347
# DQPSK errors = 0
# DQPSK Correct = 736
# Total captured samples = 35347
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 3 completed.

```

B - Case 2, Testing Slow Mode (250 Kbps) :

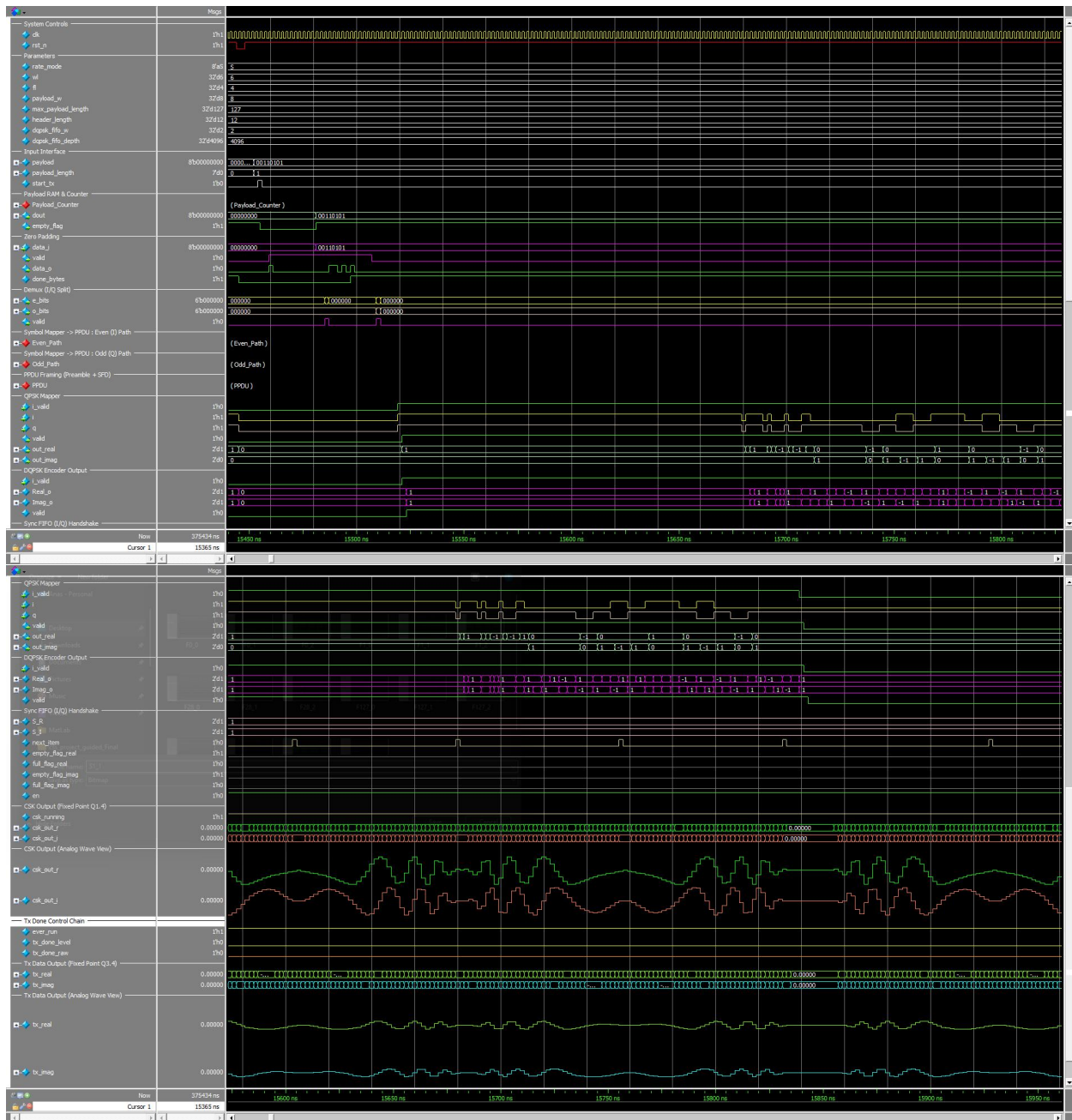
Test 1 - 0 Bytes of Payload (Slow):

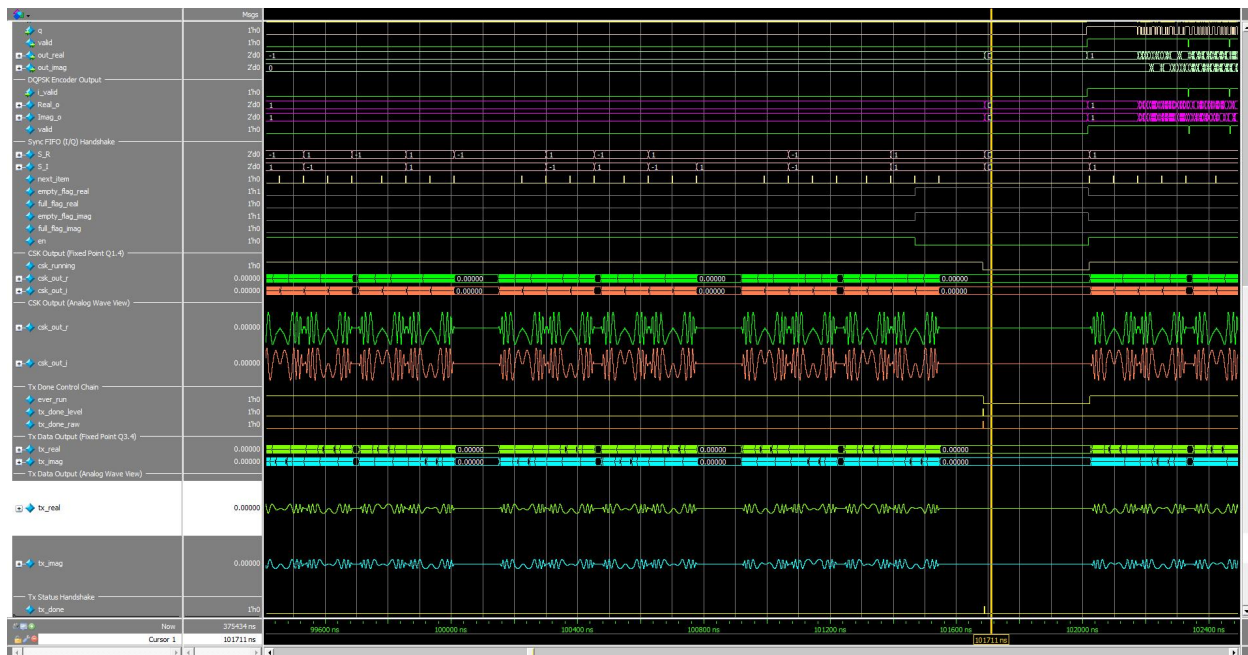




```
# Reset is OK.
# rst deasserted, starting test case 0
# Rate Mode           = S
# Payload Length      = 0
# =====
#                               15444 ns: tx_done asserted.
# TX completed. Captured 7717 samples.
# -----
# Test Case 0 Results
# Payload Length = 0
# QPSK errors = 0
# QPSK Correct = 160
# Interleaver errors = 0
# Interleaver Correct = 64
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 2
# Zero Padding errors = 0
# Zero Padding Correct = 24
# Demux errors = 0
# Demux Correct = 2
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 7717
# IMAG Correct = 7717
# DQPSK errors = 0
# DQPSK Correct = 160
# Total captured samples = 7717
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 0 completed.
```

Test 2 - 1 Bytes of Payload (Slow):





Reset is OK.

rst deasserted, starting test case 1

Rate Mode = S

Payload Length = 1

=====

30892 ns: tx_done asserted.

TX completed. Captured 7718 samples.

Test Case 1 Results

Payload Length = 1

QPSK errors = 0

QPSK Correct = 160

Interleaver errors = 0

Interleaver Correct = 64

Symbol Mapper errors = 0

Symbol Mapper Correct = 2

Zero Padding errors = 0

Zero Padding Correct = 24

Demux errors = 0

Demux Correct = 2

REAL errors = 0

IMAG errors = 0

REAL Correct = 7718

IMAG Correct = 7718

DQPSK errors = 0

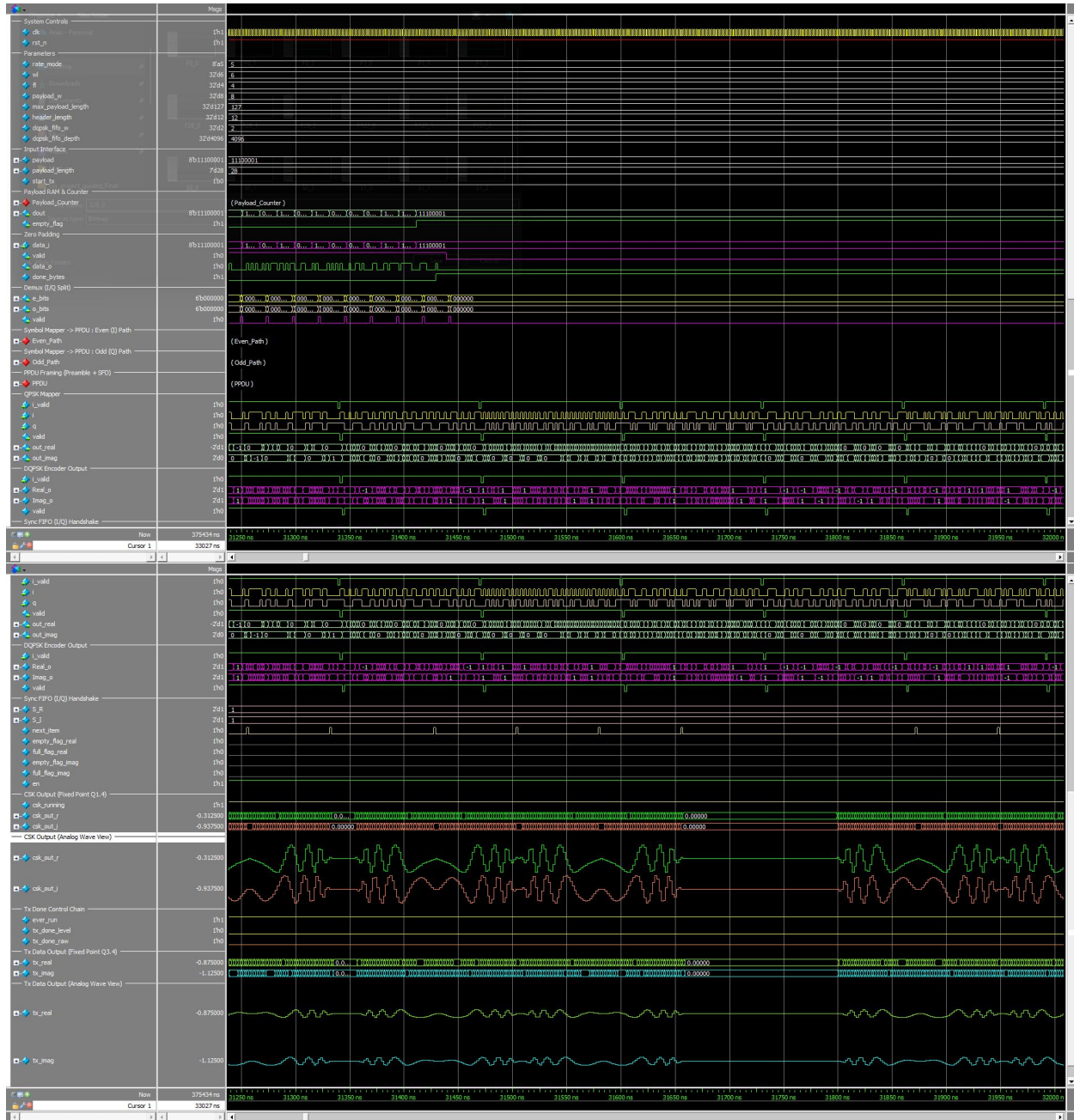
DQPSK Correct = 160

Total captured samples = 7718

***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****

test cases no = 1 completed.

Test 3 - 28 Bytes of Payload (Slow):





Reset is OK.

rst deasserted, starting test case 2

Rate Mode = 5

Payload Length = 28

=====

101688 ns: tx_done asserted.

TX completed. Captured 35365 samples.

Test Case 2 Results

Payload Length = 28

QPSK errors = 0

QPSK Correct = 736

Interleaver errors = 0

Interleaver Correct = 640

Symbol Mapper errors = 0

Symbol Mapper Correct = 20

Zero Padding errors = 0

Zero Padding Correct = 240

Demux errors = 0

Demux Correct = 20

REAL errors = 0

IMAG errors = 0

REAL Correct = 35365

IMAG Correct = 35365

DQPSK errors = 0

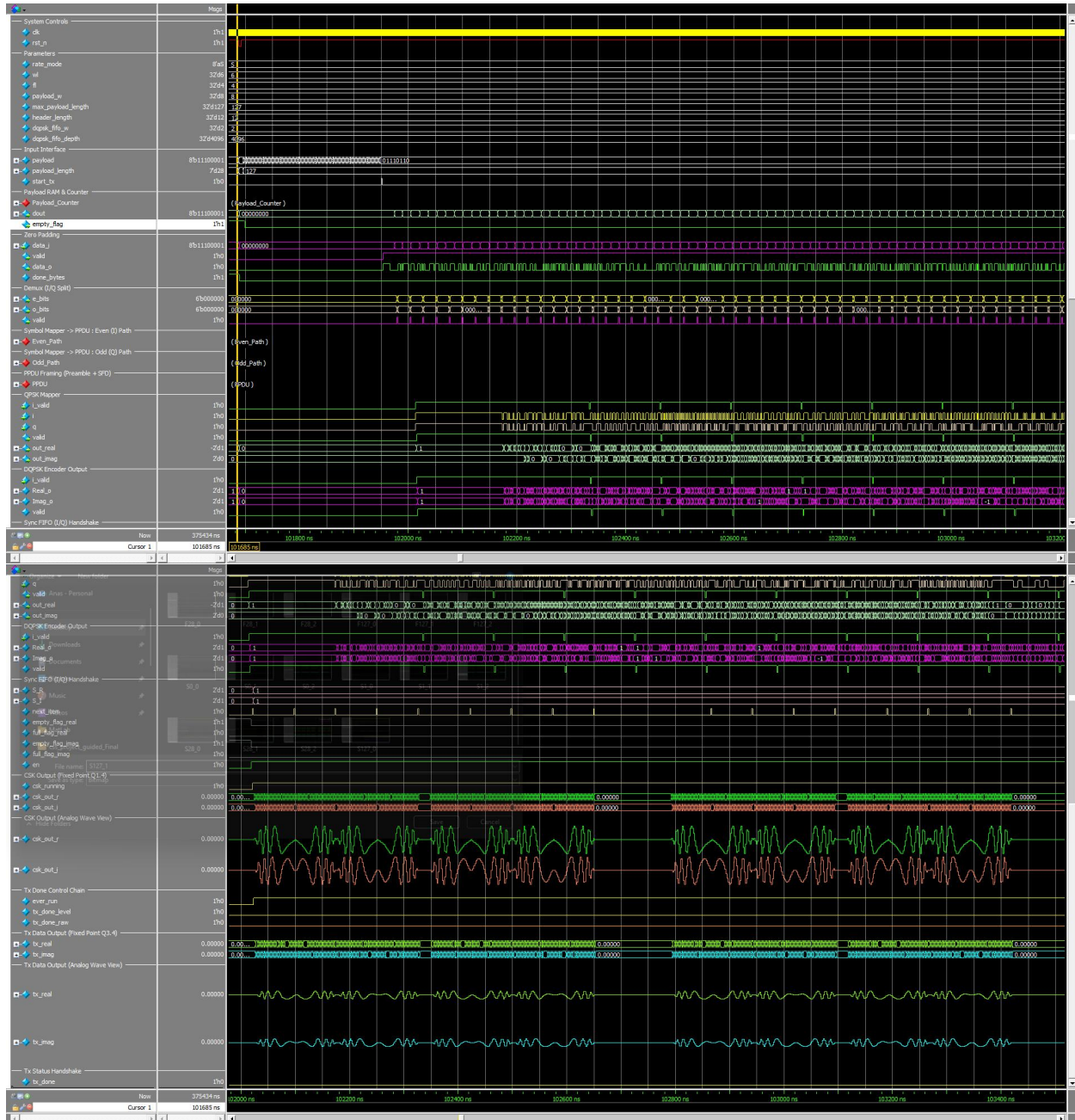
DQPSK Correct = 736

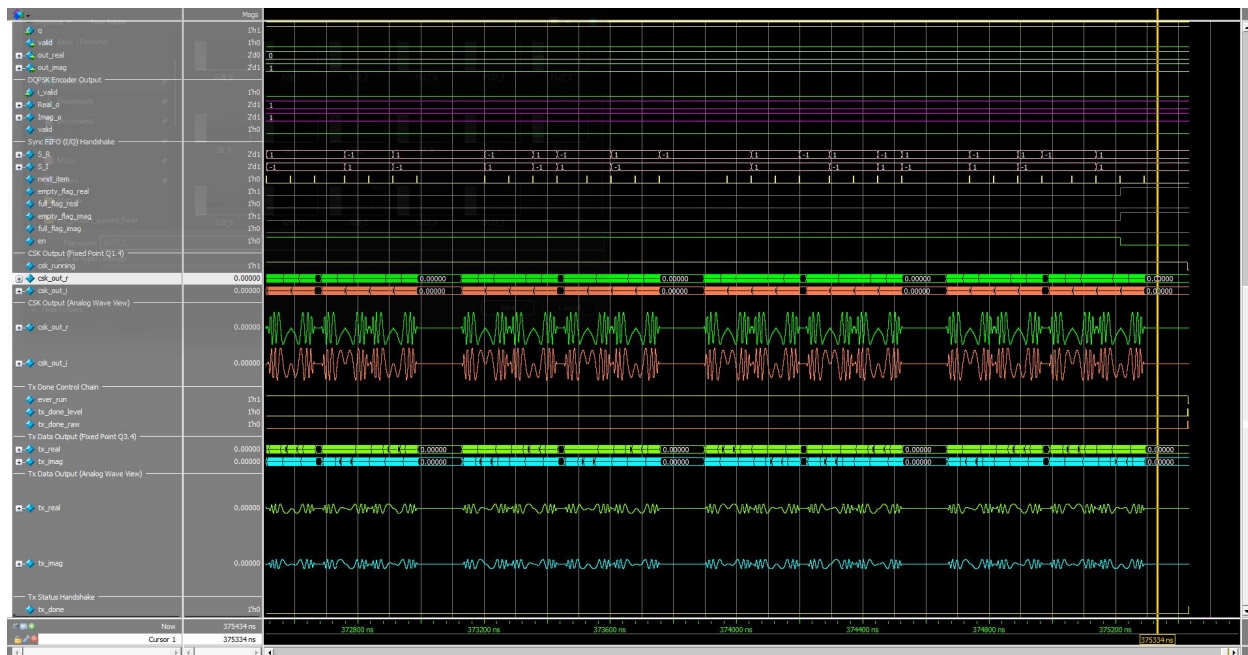
Total captured samples = 35365

***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****

test cases no = 2 completed.

Test 4 - 127 Bytes of Payload (SLOW):





```

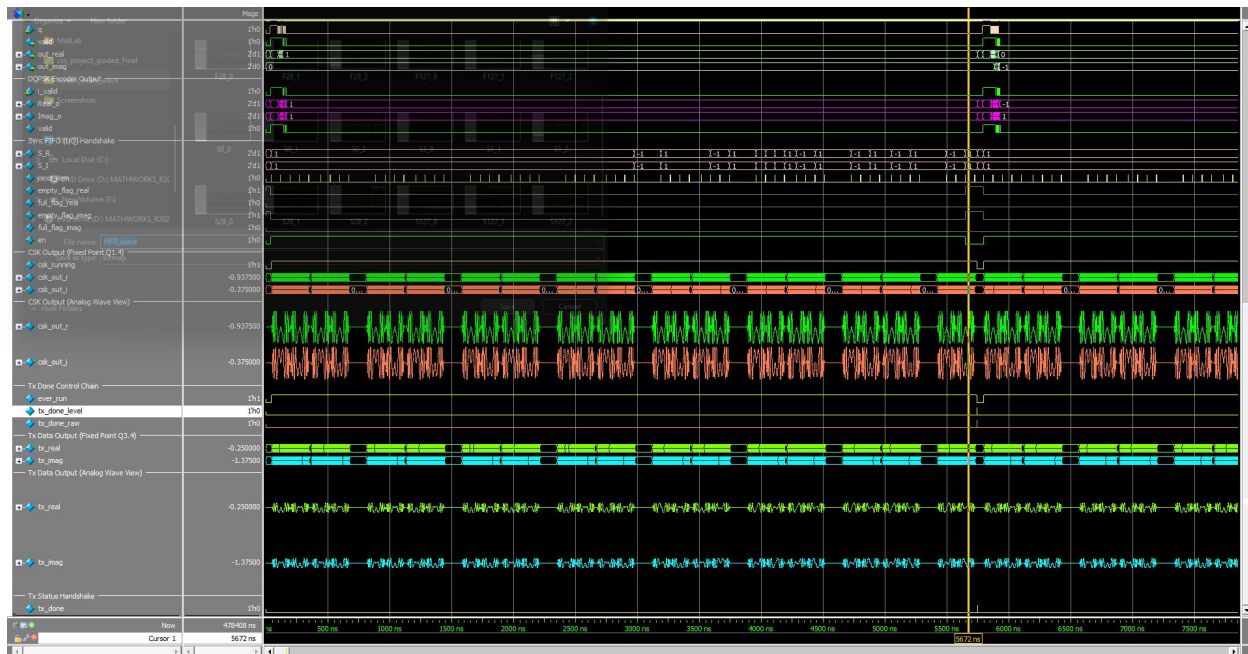
Reset is OK.
rst deasserted, starting test case 3
Rate Mode      = S
Payload Length = 127
=====
375434 ns: tx_done asserted.
TX completed. Captured 136741 samples.
-----

Test Case 3 Results
Payload Length = 127
QPSK errors = 0
QPSK Correct = 2848
Interleaver errors = 0
Interleaver Correct = 2752
Symbol Mapper errors = 0
Symbol Mapper Correct = 86
Zero Padding errors = 0
Zero Padding Correct = 1032
Demux errors = 0
Demux Correct = 86
REAL errors = 0
IMAG errors = 0
REAL Correct = 136741
IMAG Correct = 136741
DQPSK errors = 0
DQPSK Correct = 2848
Total captured samples = 136741
-----
***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
test cases no = 3 completed.

```

C - Case 3.1, Testing Hyprid Mode (1Mbps or 250Kbps):

Test 1 - 0 Bytes of Payload - Mode chosen -> Fast (1Mbps):



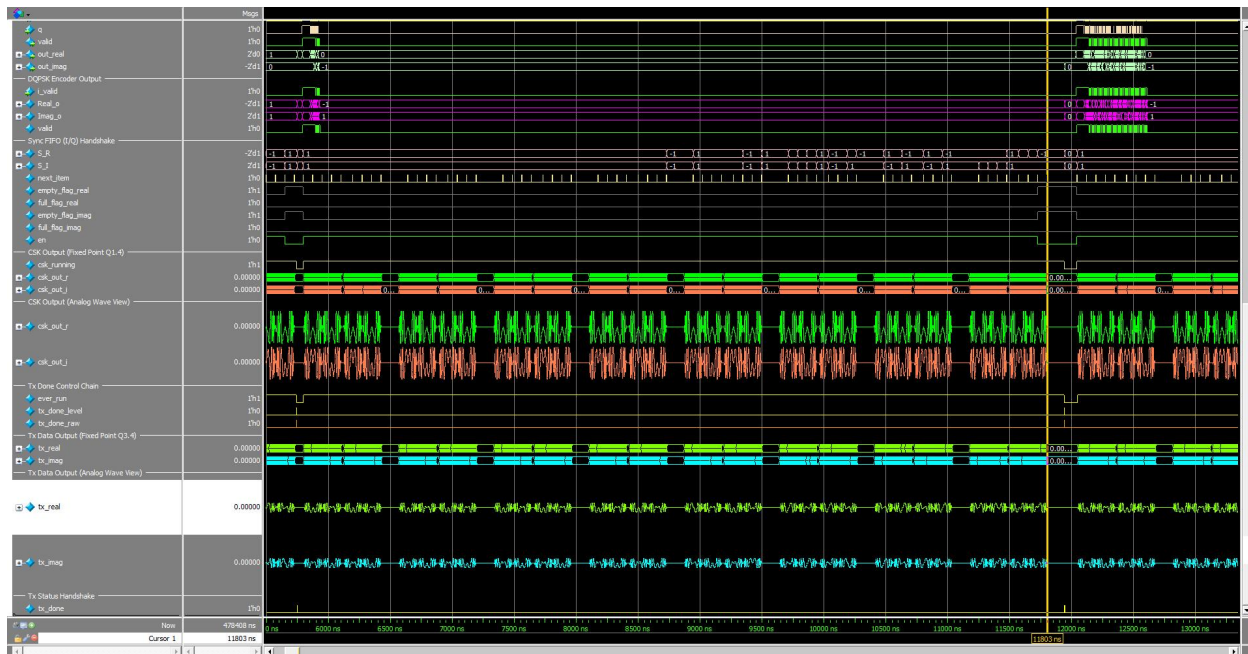
```

Reset is OK.
rst deasserted, starting test case 0
Rate Mode      = H
Payload Length = 0
=====
                    5748 ns: tx_done asserted.
TX completed. Captured 2869 samples.
-----

Test Case 0 Results
Payload Length = 0
QPSK errors = 0
QPSK Correct = 60
Interleaver errors = 0
Interleaver Correct = 12
Symbol Mapper errors = 0
Symbol Mapper Correct = 3
Zero Padding errors = 0
Zero Padding Correct = 18
Demux errors = 0
Demux Correct = 3
REAL errors = 0
IMAG errors = 0
REAL Correct = 2869
IMAG Correct = 2869
DQPSK errors = 0
DQPSK Correct = 60
Total captured samples = 2869
-----
***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
test cases no = 0 completed.

```

Test 2 - 1 Bytes of Payload - Mode chosen -> Fast (1Mbps):



```

Reset is OK.
First deasserted, starting test case 1
Rate Mode      = H
Payload Length = 1

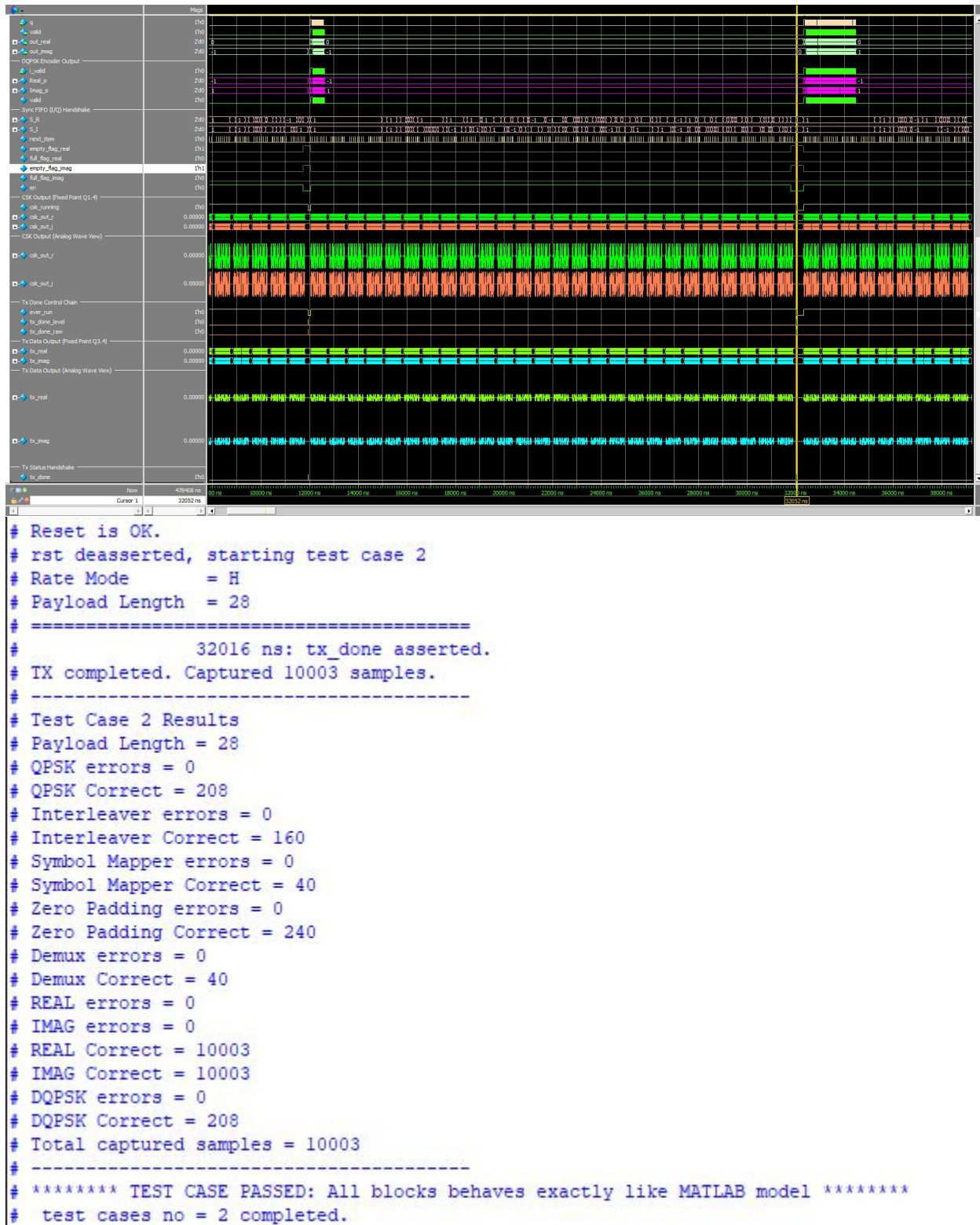
=====
11944 ns: tx_done asserted.
TX completed. Captured 3092 samples.
-----

Test Case 1 Results
Payload Length = 1
QPSK errors = 0
QPSK Correct = 64
Interleaver errors = 0
Interleaver Correct = 16
Symbol Mapper errors = 0
Symbol Mapper Correct = 4
Zero Padding errors = 0
Zero Padding Correct = 24
Demux errors = 0
Demux Correct = 4
REAL errors = 0
IMAG errors = 0
REAL Correct = 3092
IMAG Correct = 3092
DQPSK errors = 0
DQPSK Correct = 64
Total captured samples = 3092
-----

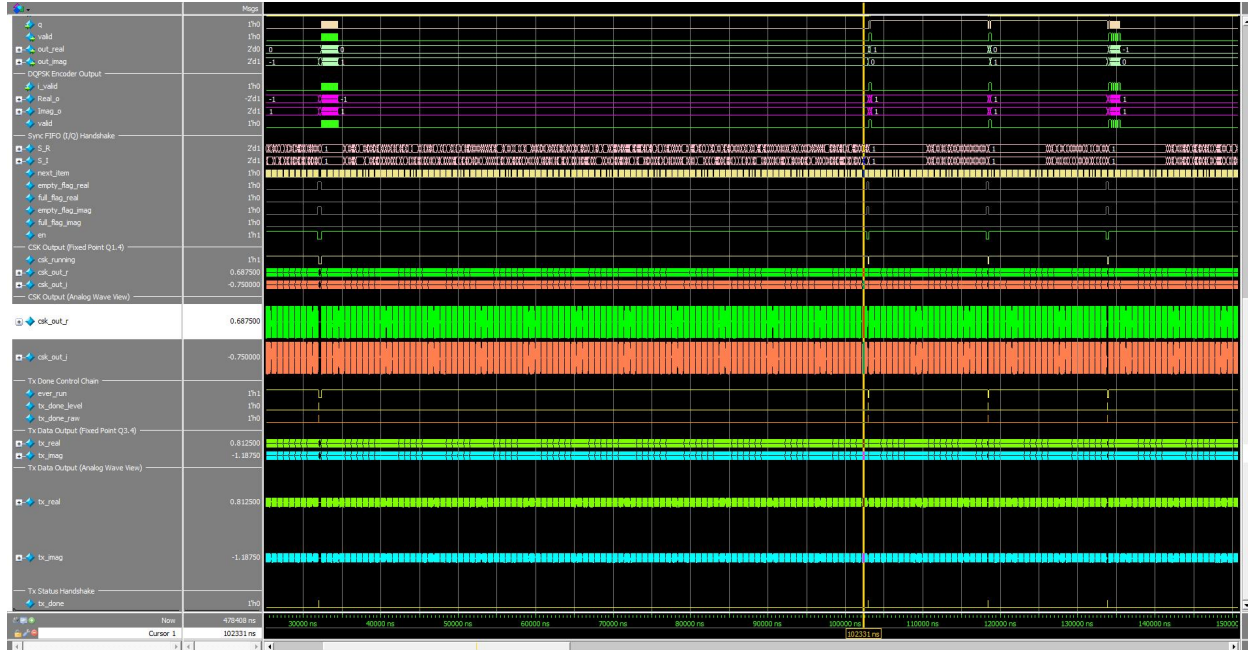
***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
test cases no = 1 completed.

```

Test 3 - 28 Bytes of Payload - Mode chosen -> Fast (1Mbps):



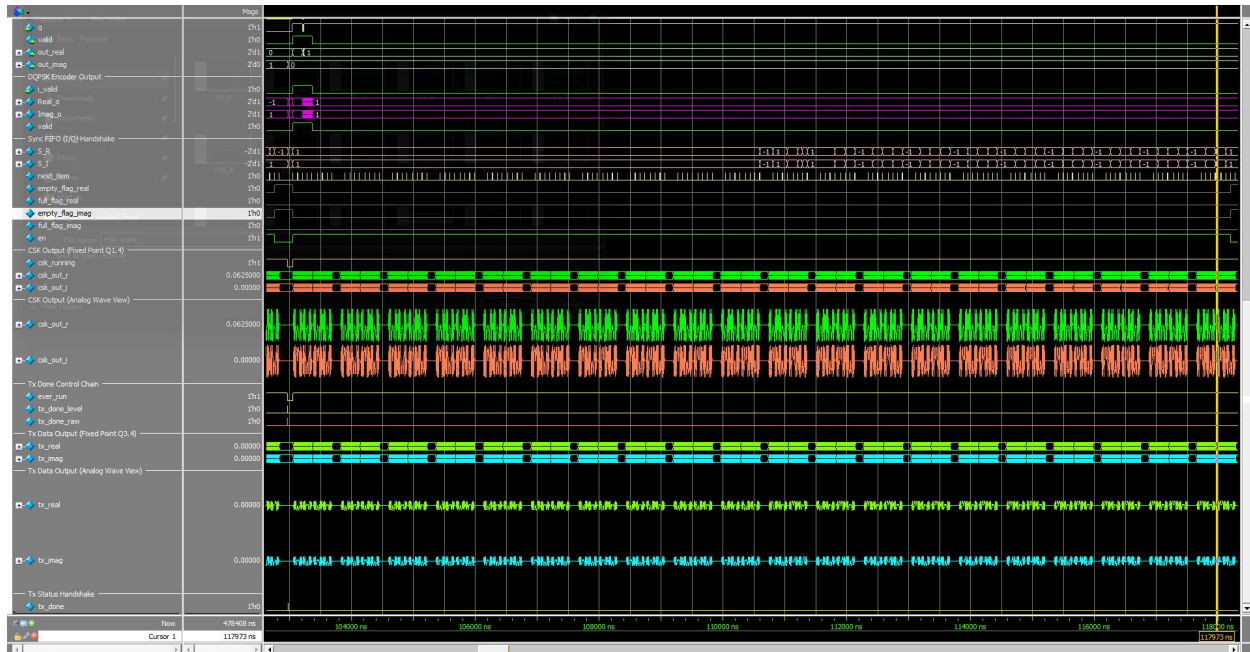
Test 4 - 127 Bytes of Payload - Mode chosen -> Fast (1Mbps):



```
# Reset is OK.
# rst deasserted, starting test case 3
# Rate Mode      = H
# Payload Length  = 127
# =====
#                102974 ns: tx_done asserted.
# TX completed. Captured 35347 samples.
# -----
# Test Case 3 Results
# Payload Length = 127
# QPSK errors = 0
# QPSK Correct = 736
# Interleaver errors = 0
# Interleaver Correct = 688
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 172
# Zero Padding errors = 0
# Zero Padding Correct = 1032
# Demux errors = 0
# Demux Correct = 172
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 35347
# IMAG Correct = 35347
# DQPSK errors = 0
# DQPSK Correct = 736
# Total captured samples = 35347
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 3 completed.
```

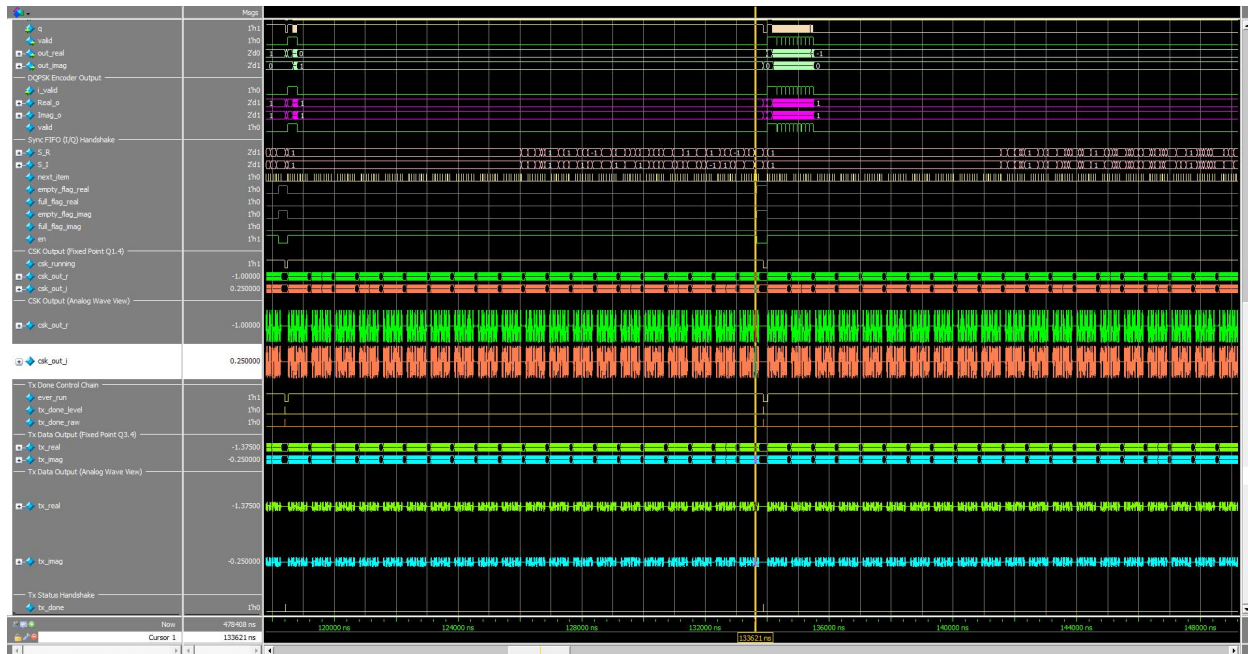
C - Case 3.2, Testing Hyprid Mode :

Test 1 - 0 Bytes of Payload - Mode chosen -> Slow (250Kbps):



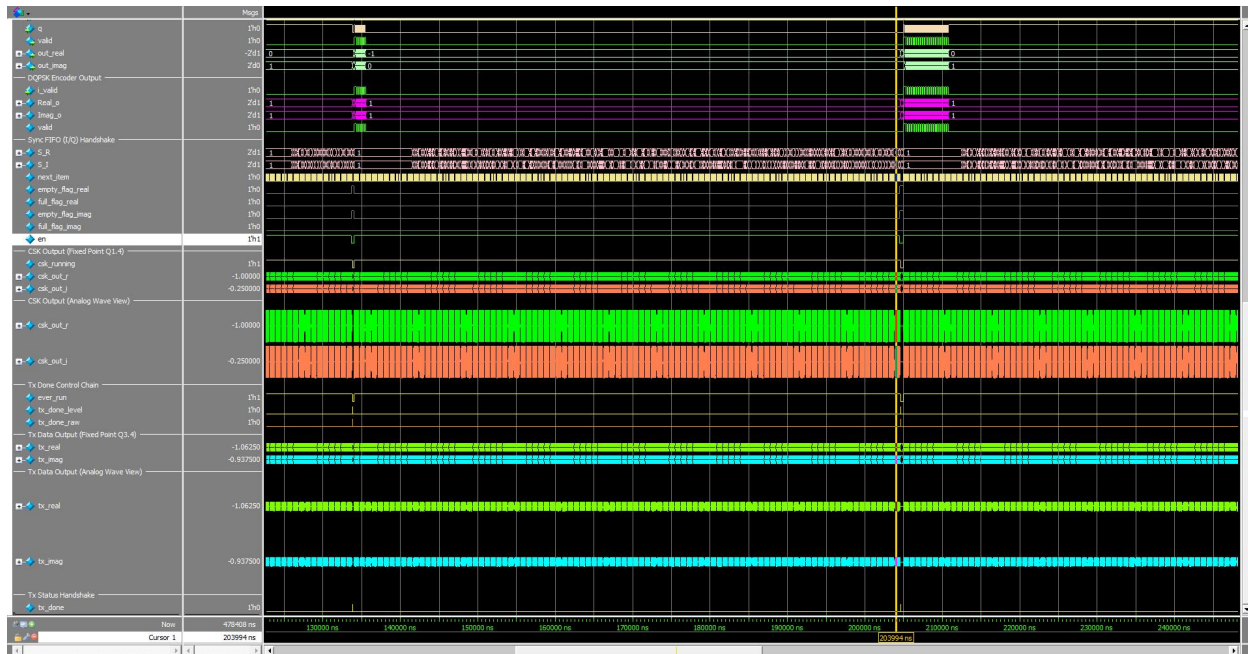
```
# Reset is OK.
# rst deasserted, starting test case 4
# Rate Mode      = H
# Payload Length = 0
# =====
#                  118418 ns: tx_done asserted.
# TX completed. Captured 7716 samples.
# -----
# Test Case 4 Results
# Payload Length = 0
# QPSK errors = 0
# QPSK Correct = 160
# Interleaver errors = 0
# Interleaver Correct = 64
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 2
# Zero Padding errors = 0
# Zero Padding Correct = 24
# Demux errors = 0
# Demux Correct = 2
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 7716
# IMAG Correct = 7716
# DQPSK errors = 0
# DQPSK Correct = 160
# Total captured samples = 7716
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 4 completed.
```

Test 2 - 1 Bytes of Payload - Mode chosen -> Slow (250Kbps):



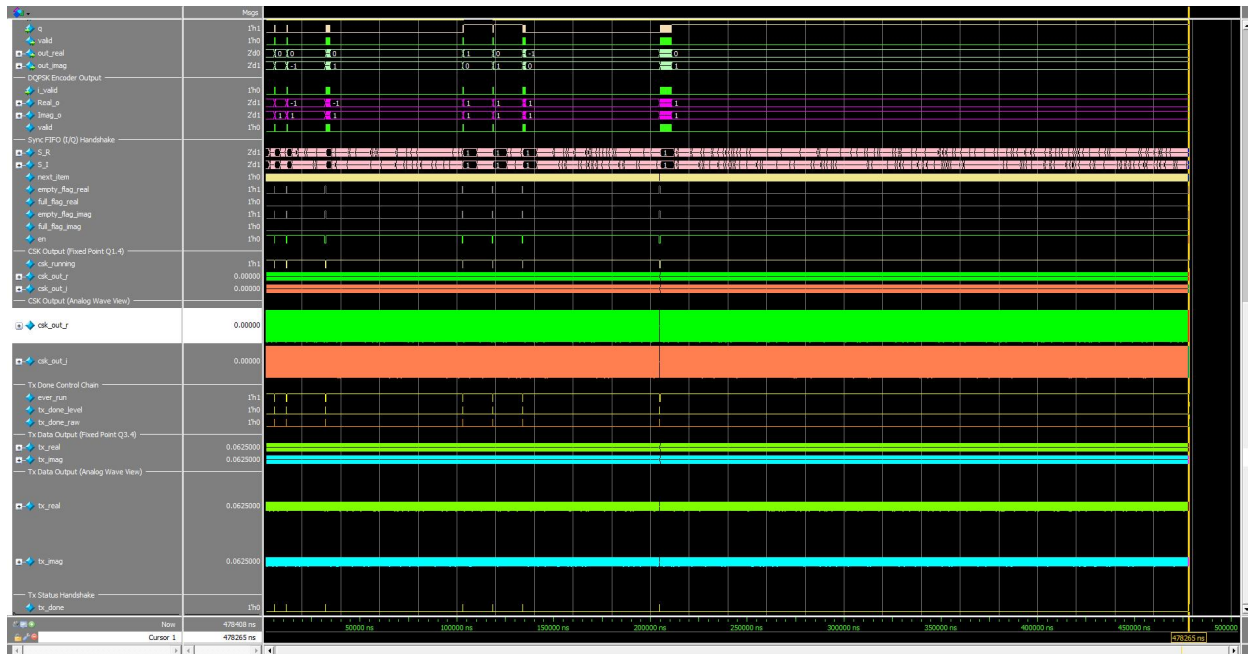
```
# rst deasserted, starting test case 5
# Rate Mode      = H
# Payload Length = 1
# =====
#               133866 ns: tx_done asserted.
# TX completed. Captured 7717 samples.
# -----
# Test Case 5 Results
# Payload Length = 1
# QPSK errors = 0
# QPSK Correct = 160
# Interleaver errors = 0
# Interleaver Correct = 64
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 2
# Zero Padding errors = 0
# Zero Padding Correct = 24
# Demux errors = 0
# Demux Correct = 2
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 7717
# IMAG Correct = 7717
# DQPSK errors = 0
# DQPSK Correct = 160
# Total captured samples = 7717
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 5 completed.
```

Test 3 - 28 Bytes of Payload - Mode chosen -> Slow (250Kbps):



```
# Reset is OK.
# rst deasserted, starting test case 6
# Rate Mode      = H
# Payload Length = 28
# =====
#                               204662 ns: tx_done asserted.
# TX completed. Captured 35364 samples.
# -----
# Test Case 6 Results
# Payload Length = 28
# QPSK errors = 0
# QPSK Correct = 736
# Interleaver errors = 0
# Interleaver Correct = 640
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 20
# Zero Padding errors = 0
# Zero Padding Correct = 240
# Demux errors = 0
# Demux Correct = 20
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 35364
# IMAG Correct = 35364
# DQPSK errors = 0
# DQPSK Correct = 736
# Total captured samples = 35364
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
# test cases no = 6 completed.
```

Test 4 - 127 Bytes of Payload - Mode chosen -> Slow (250Kbps):



```
# Reset is OK.
# rst deasserted, starting test case 7
# Rate Mode      = H
# Payload Length = 127
# =====
#               478408 ns: tx_done asserted.
# TX completed. Captured 136740 samples.
# -----
# Test Case 7 Results
# Payload Length = 127
# QPSK errors = 0
# QPSK Correct = 2848
# Interleaver errors = 0
# Interleaver Correct = 2752
# Symbol Mapper errors = 0
# Symbol Mapper Correct = 86
# Zero Padding errors = 0
# Zero Padding Correct = 1032
# Demux errors = 0
# Demux Correct = 86
# REAL errors = 0
# IMAG errors = 0
# REAL Correct = 136740
# IMAG Correct = 136740
# DQPSK errors = 0
# DQPSK Correct = 2848
# Total captured samples = 136740
# -----
# ***** TEST CASE PASSED: All blocks behaves exactly like MATLAB model *****
```

4.4 Results Summary

Overview

Verification was performed across three rate-mode configurations — 1 Mb/s (Fast, "F"), 250 kb/s (Slow, "S"), and hybrid ("H") — each exercised against multiple payload-length test cases (0, 1, and 28 bytes shown here, with the same methodology applied at the maximum 127-byte length). Every test case reported a full **PASS**, with the self-checking testbench confirming that all blocks behave exactly as the MATLAB golden model across every captured sample.

For every passing case, per-stage error counters — QPSK mapper, interleaver, symbol mapper, zero padding, demux, REAL/IMAG datapaths, and DQPSK encoder — all reported zero errors, with correct-sample counts matching the golden reference exactly.

Verification Approach and Process

Initial full-chain integration did not produce correct behavior, and observing only the final tx_real/tx_imag outputs gave no useful diagnostic information when a mismatch occurred — a failure at the output could originate anywhere upstream. Verification therefore proceeded stage-by-stage, working backward from the output:

1. Each stage's output was compared against the corresponding MATLAB golden reference, starting from the final transmitted samples and working back through the DQPSK encoder, QPSK mapper, interleaver, symbol mapper, demux, and zero-padding stages.
2. When a stage's output matched the golden reference, verification moved forward to check the next stage in the chain.
3. When a mismatch was found, the faulty stage was isolated first from the error/correct counters and log files where the discrepancy was directly traceable, and where the log alone was insufficient, by direct waveform inspection to identify the specific cycle and condition causing the divergence.
4. Once isolated, the root cause was identified in the RTL, corrected, and the affected stage was re-verified in isolation before re-running the full chain.
5. This cycle repeated until every stage passed independently and the full chain matched the golden model end-to-end.

This process was applied first to the Fast (1 Mb/s) mode, then Slow (250 kb/s), and finally the Hybrid mode, with each mode re-verified against the full set of payload-length test cases to confirm the RTL meets specification under every supported rate and frame-size configuration — not only under the conditions where a bug happened to be found.

Key Observations from Verification

- **Pipeline alignment is the dominant class of issue in a chain this deep.** Because the CSK generator, DQPSK encoder, and complex multiplier each introduce their own registered latency, mismatches were often not a computational error in a given block, but a one-or-few-cycle misalignment between a block's output becoming valid and the corresponding sample from an upstream or downstream block being sampled at the wrong time. This is consistent with the explicit pipeline-alignment logic documented in the CSK generator (`first_fetch/boundary_d`), which exists specifically to keep the DQPSK symbol and chirp sample in lockstep at each subchirp boundary — a mechanism introduced as a direct consequence of this stage-by-stage debugging process.

- **Waveform inspection was necessary, not optional, for timing-related mismatches.** Log files reliably localized *which* stage and *which* sample index disagreed with the golden model, but the *why* — typically a validity signal asserting a cycle early or late relative to its associated data — required direct waveform correlation across the CSK output, DQPSK encoder output, and sync FIFO handshake signals to confirm.
- **Re-verification after each fix was mandatory,** since a fix to a timing issue in one stage could shift the validity window seen by the next stage; a stage that passed before a fix was not assumed to still pass afterward without re-running its check.
- **All three rate modes required independent verification** rather than assuming correctness in one mode implied correctness in another, since the hybrid mode in particular exercises additional logic (rate detection from start_tx pulse timing) that has no equivalent in the fixed-rate Fast and Slow builds.

Conclusion

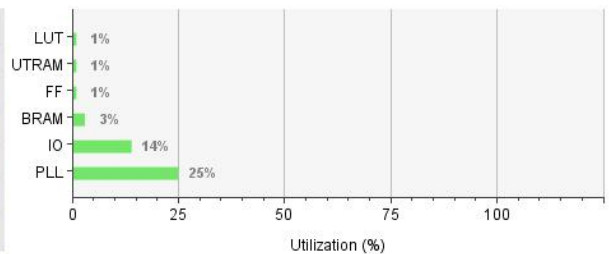
Following this iterative, stage-isolating verification methodology, all identified RTL defects were corrected at their root cause rather than patched at the output, and the design was confirmed — across Fast, Slow, and Hybrid rate modes and across the full range of tested payload lengths — to match the MATLAB golden reference model bit-exactly at every stage of the transmit chain.

5.FPGA Resource Utilization, Timing Results & Known Limitations

5.1 Resource Utilization

Name	Slice LUTs (53200)	Slice Registers (106400)	Block RAM Tile (140)	Bonded IOB (200)	BUFGCTRL (32)	PLLE2_ADV (4)
zigbee_fpga	685	696	4.5	27	2	1
DEMUX (e_o_demux)	57	32	0	0	0	0
DQPSK_CSK (dqpsk_...)	115	133	2	0	0	0
my_controller (controll...)	7	3	0	0	0	0
payload_ram (FIFO_m...)	18	16	0.5	0	0	0
pll_0 (clk_wiz_0)	1	0	0	0	2	1
QPSK (QPSK_mapper)	4	5	0	0	0	0
symbol_mapper_to_p...	377	432	2	0	0	0
u_uart_rx (uart_rx)	45	40	0	0	0	0
ZeroPadding (zeropad...)	44	27	0	0	0	0

Resource	Utilization	Available	Utilization %
LUT	685	53200	1.29
LUTRAM	14	17400	0.08
FF	696	106400	0.65
BRAM	4.50	140	3.21
IO	27	200	13.50
PLL	1	4	25.00



Resource	Used / Available (Utilization %)
LUTs	1.29%
Flip-Flops	0.65%
Block RAM / ROM	3.21%

5.2 Power results

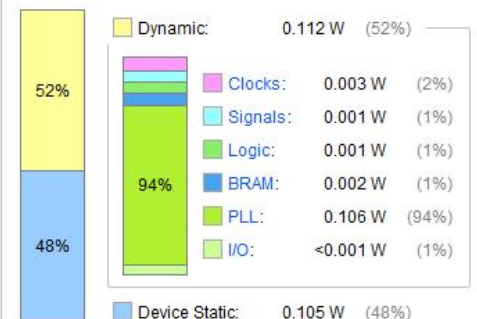
Summary

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: 0.217 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 27.5°C
 Thermal Margin: 57.5°C (4.8 W)
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



5.3 Timing Results

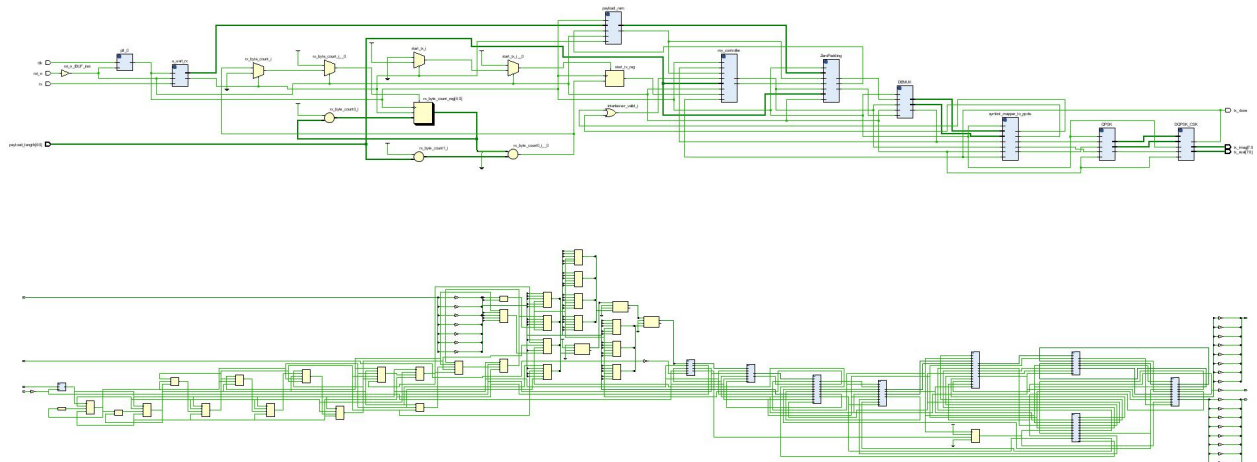
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 25.023 ns	Worst Hold Slack (WHS): 0.070 ns	Worst Pulse Width Slack (WPWS): 2.633 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1610	Total Number of Endpoints: 1610	Total Number of Endpoints: 728

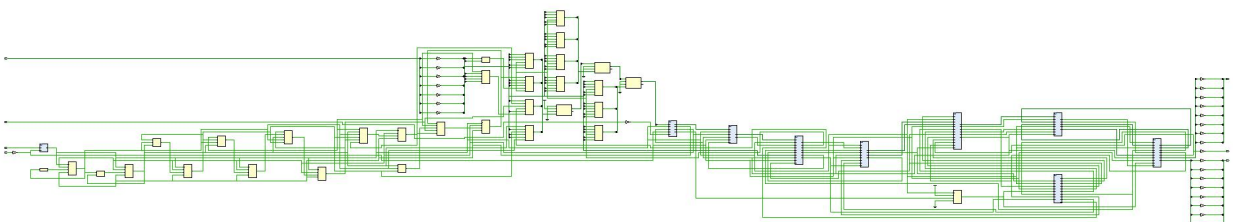
All user specified timing constraints are met.

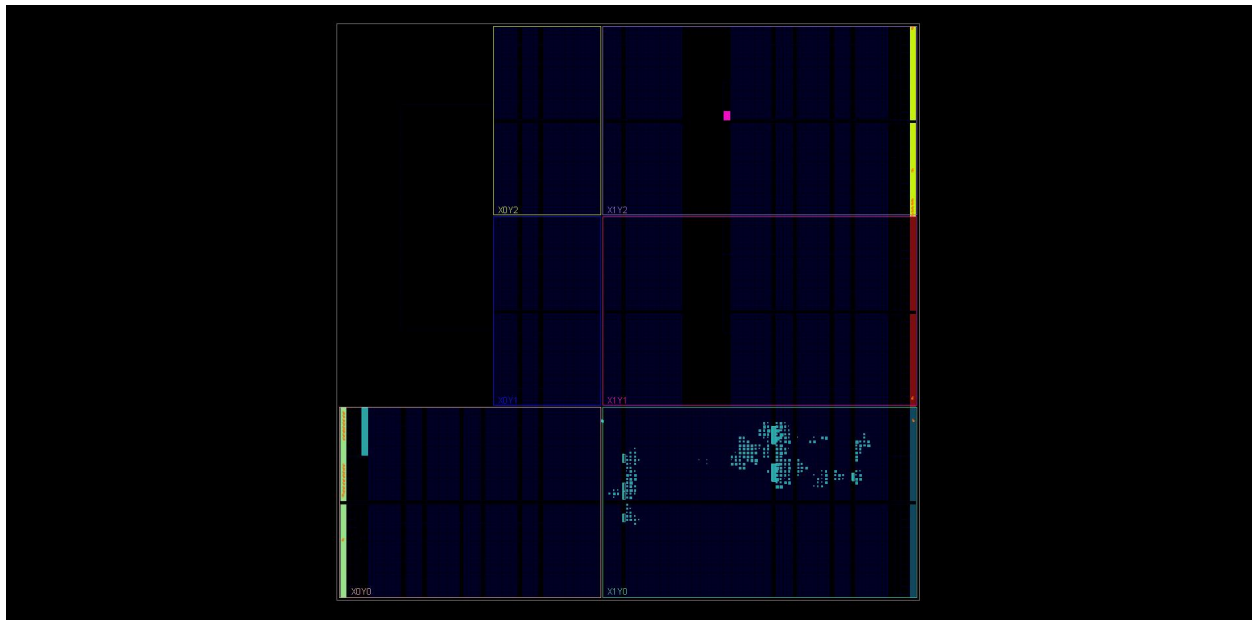
Metric	Result
Target clock frequency	32 Mhz
Achieved Fmax	32 Mhz
Worst setup slack	24.308
Worst hold slack	0.033
Critical path	from symbol_mapper_to_ppdu/u_symbol_mapper_odd/gen_mapper_hybrid.o_data_reg[8]/C to : symbol_mapper to ppdu/u_sync_fifo_odd/mem_reg/DIADI[8]

5.4 Schematics



5.4 Implementation





5.6 Timing constraints

```
##### Timing Constraints #####

# Primary Clock (100 MHz)
create_clock -name clk -period 10.000 [get_ports clk]

# Generated 32 MHz Clock by PLL
create_generated_clock -name clk_32 \
    -source [get_pins pll_0/clk_in] \
    -multiply_by 8 \
    -divide_by 25 \
    [get_pins pll_0/clk_32]

# False Paths
set_false_path -from [all_inputs]
set_false_path -to [all_outputs]

# Input & Output Delays
set_input_delay -clock [get_clocks clk_32] -max 0.2 [get_ports {rx payload_length[*]}]
set_input_delay -clock [get_clocks clk_32] -min 0.1 [get_ports {rx payload_length[*]}]
set_output_delay -clock [get_clocks clk_32] -max 0.2 [all_outputs]
set_output_delay -clock [get_clocks clk_32] -min 0.1 [all_outputs]
```

5.7 Physical constraints

```
##### Physical Constraints #####

# Primary System Clock (100 MHz - Bank 13 / GCLK)
set_property PACKAGE_PIN Y9 [get_ports {clk}]

# UART RX Pin (PMOD JA - Pin Y11)
set_property PACKAGE_PIN AB6 [get_ports rx]

# Reset Button (BTNC - Bank 34, 1.8V)
set_property PACKAGE_PIN P16 [get_ports rst_n]

# 8-bit Payload Length Slide Switches (SW0 - SW7 - Bank 35, 1.8V)
set_property PACKAGE_PIN F22 [get_ports {payload_length[0]}]
set_property PACKAGE_PIN G22 [get_ports {payload_length[1]}]
set_property PACKAGE_PIN H22 [get_ports {payload_length[2]}]
set_property PACKAGE_PIN F21 [get_ports {payload_length[3]}]
set_property PACKAGE_PIN H19 [get_ports {payload_length[4]}]
set_property PACKAGE_PIN H18 [get_ports {payload_length[5]}]
set_property PACKAGE_PIN H17 [get_ports {payload_length[6]}]

# TX Done Indicator LED (LD0 - Bank 33, 3.3V)
set_property PACKAGE_PIN T22 [get_ports tx_done]

# TX Real Outputs (PMOD JB - 3.3V)
set_property PACKAGE_PIN W12 [get_ports {tx_real[0]}]
set_property PACKAGE_PIN W11 [get_ports {tx_real[1]}]
set_property PACKAGE_PIN V10 [get_ports {tx_real[2]}]
set_property PACKAGE_PIN W8 [get_ports {tx_real[3]}]
set_property PACKAGE_PIN V12 [get_ports {tx_real[4]}]
set_property PACKAGE_PIN W10 [get_ports {tx_real[5]}]
set_property PACKAGE_PIN V9 [get_ports {tx_real[6]}]
set_property PACKAGE_PIN V8 [get_ports {tx_real[7]}]

# TX Imaginary Outputs (PMOD JA - 3.3V)
set_property PACKAGE_PIN Y11 [get_ports {tx_imag[0]}]
set_property PACKAGE_PIN AA11 [get_ports {tx_imag[1]}]
set_property PACKAGE_PIN Y10 [get_ports {tx_imag[2]}]
set_property PACKAGE_PIN AA9 [get_ports {tx_imag[3]}]
set_property PACKAGE_PIN AB11 [get_ports {tx_imag[4]}]
set_property PACKAGE_PIN AB10 [get_ports {tx_imag[5]}]
set_property PACKAGE_PIN AB9 [get_ports {tx_imag[6]}]
set_property PACKAGE_PIN AA8 [get_ports {tx_imag[7]}]

# Explicit IO Standards (Targeting ports individually to prevent bank-object lookup failures)
set_property IOSTANDARD LVCMOS33 [get_ports {clk rx tx_done tx_real[*] tx_imag[*]}]
set_property IOSTANDARD LVCMOS18 [get_ports {rst_n payload_length[*]}]
```

5.8 Uart protocol & Pin Allocation

The physical implementation of the Zigbee transceiver on the ZedBoard Zynq-7000 FPGA is mapped across the Programmable Logic (PL) I/O headers. To ensure signal integrity and prevent pin contention between the high-speed data buses and control interfaces, the physical constraints are structured as follows:

- **UART Interface (rx):** Configured on **PMOD JC (Pin AB6)**, receiving incoming serial payload bytes at a standard baud rate of 115,200 driven by the 32 MHz internal clock domain.
- **In-Phase Component (tx_real[7:0]):** Mapped to the **PMOD JB** header (W12 through V8), providing an 8-bit parallel output bus for the modulated real data path under a 3.3V LVCMOS standard.
- **Quadrature Component (tx_imag[7:0]):** Mapped across **PMOD JA** pins (Y11 through AA8), isolating the imaginary data path on a separate physical connector to avoid bus collisions.
- **Payload Length Configuration (payload_length[6:0]):** Controlled via 7 onboard slide switches (F22 to H17) on Bank 35 (1.8V), allowing real-time configuration of the packet byte count up to the maximum payload limit of 127 bytes.
- **Clock and Reset Management:** Driven by the primary onboard 100 MHz oscillator (Y9) and synchronized via an internal PLL clock wizard (clk_wiz_0) generating the clk_32 domain, with the system reset linked to push-button P16.
- **Status Indication (tx_done):** Tied to LED T22 to visually indicate the completion of a transmission cycle.

6. ASIC Resource Utilization, Timing Results & Known Limitations

6.1 Synthesis

The Targeted Technology was 90nm But, due to the presence of a big rom in the design and the absence of memory macros we decided to scale down the Technology to 32nm of flavor HVT to solve setup violations.

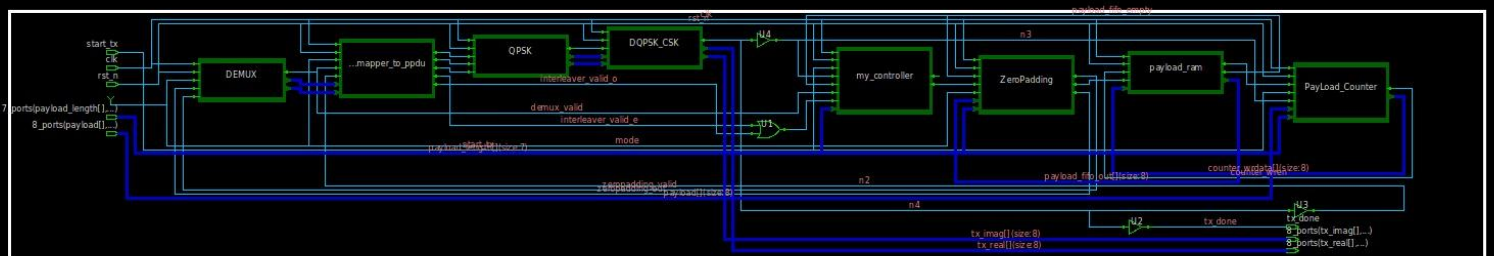
In this Report ASIC Flow is only run on Fast Mode (1Mbps) in a Single Mode Single Corner.

The Used Clock Was 32MHZ as spec and with these listed constraints:

- 3% Setup Uncertainty
- 2% Hold Uncertainty
- 20% of clock period input/output delay
- 10pf output Load
- 3.6 max transition
- 100pf max capacitance
- 10 max fanout
- 50% duty cycle

We obtained the following Results:

Spec	Value
Area	341314.002045
Setup	MET with no Violations
Combinational Cells	52956
Sequencial Cells	18526



Top Wrapper

6.2 Formal Verification

Formal Verification had to be run between the generated netlist of synthesis and the original rtl and it succeeded.

The screenshot shows the Formality (R) Console interface. The top status bar indicates "Verification Succeeded". Below this, the reference and implementation paths are listed: Reference: ref:/WORK/zigbee and Implementation: imp:/WORK/zigbee. The console is divided into several tabs: 0. Guid., 1. Ref., 2. Impl., 3. Setup, 4. Match, 5. Verify, and 6. Debug. The main window displays a table of failing points, which is currently empty, indicating that no failing points were found during the verification process.

Failing Points	Passing Points	Aborted Points	Unverified Points	Probe Points	Analyses	Loops
1	OFF	DEMUX_ie_bits_reg[0]				
2	OFF	DEMUX_ie_bits_reg[1]				
3	OFF	DEMUX_ie_bits_reg[2]				
4	OFF	DEMUX_ie_bits_reg_reg[0]				
5	OFF	DEMUX_ie_bits_reg_reg[1]				
6	OFF	DEMUX_ie_bits_reg_reg[2]				
7	OFF	DEMUX_ie_counter_reg[0]				
8	OFF	DEMUX_ie_counter_reg[1]				
9	OFF	DEMUX_ie_bits_reg[0]				
10	OFF	DEMUX_ie_bits_reg[1]				
11	OFF	DEMUX_ie_bits_reg[2]				
12	OFF	DEMUX_ie_bits_reg_reg[0]				
13	OFF	DEMUX_ie_bits_reg_reg[1]				
14	OFF	DEMUX_ie_bits_reg_reg[2]				

6.3 PnR

6.3.1 NDM

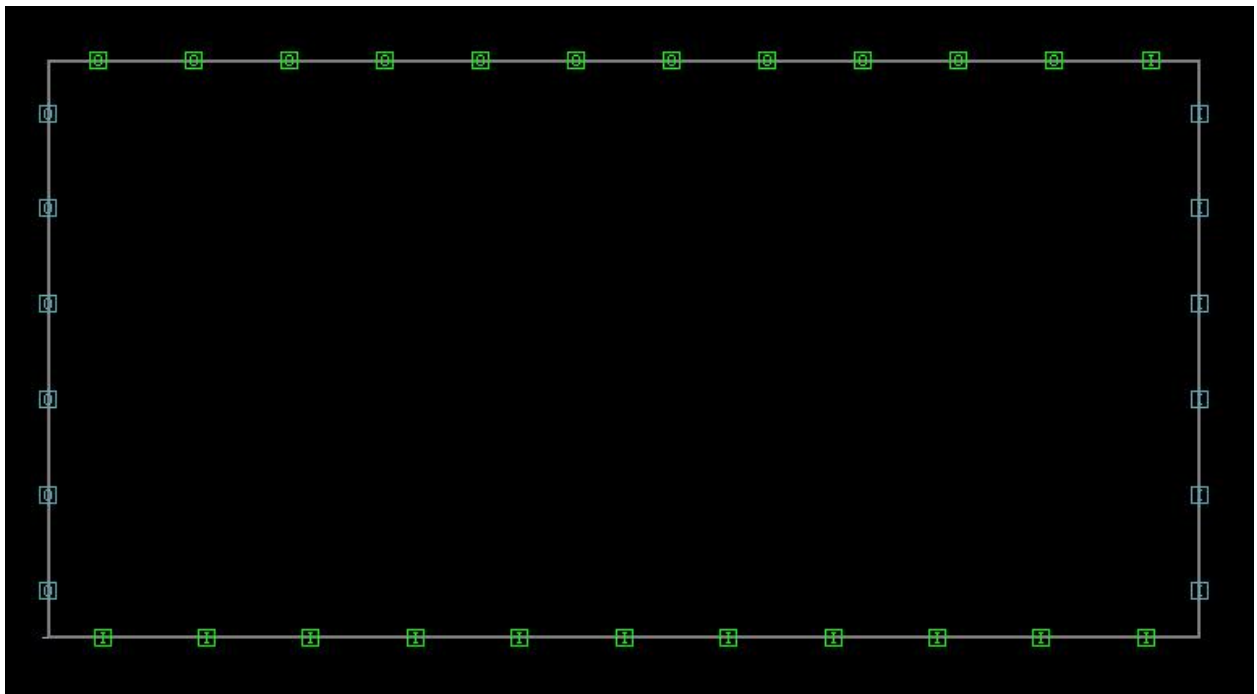
The PDK of 32 nm already had its own ndm so it was not necessary to create it.

6.3.2 Design Library

The Output Verified Netlist is imported with NDM and other Necessary files and generated the Design Library

6.3.3 Floor Plan

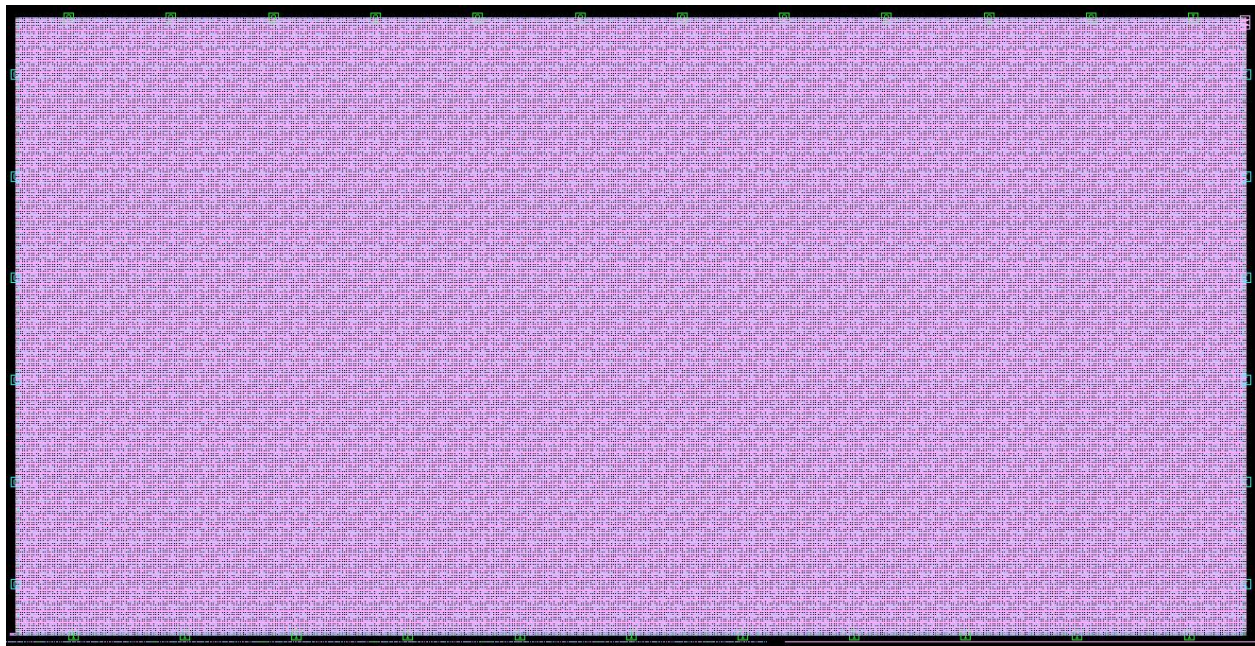
Due to design complexity, we sacrificed the area and made a utilization of 20%



6.3.4 Power Plan

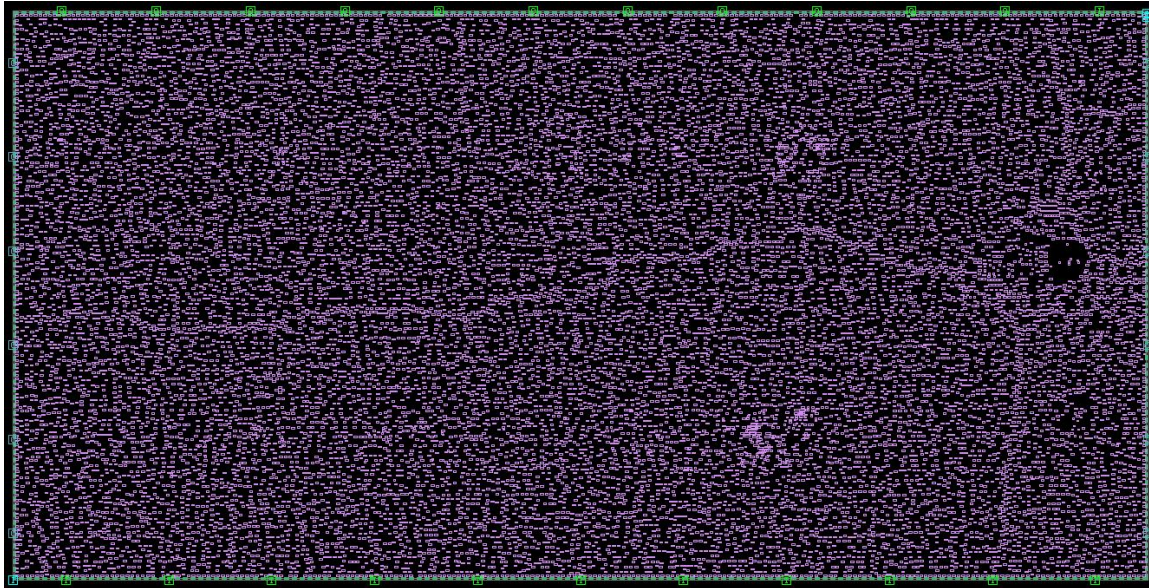
Added a Power Network with the Following Parameters:

parameter	Value
width	2
space	2
pitch	$2 * \text{vertical_space} + 2 * \text{vertical_width}$
offset	0
rail_width	0.06



6.3.4 Placement

Placed all Design Cells with 15 sparecells

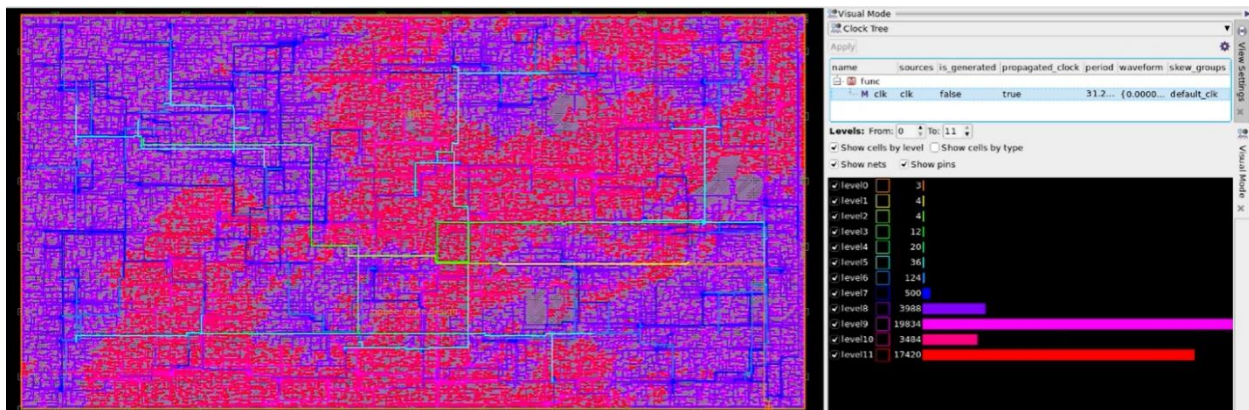


6.3.5 CTS

Ran CTS To solve Hold Violations and eliminate clock skew with the listed parameters:

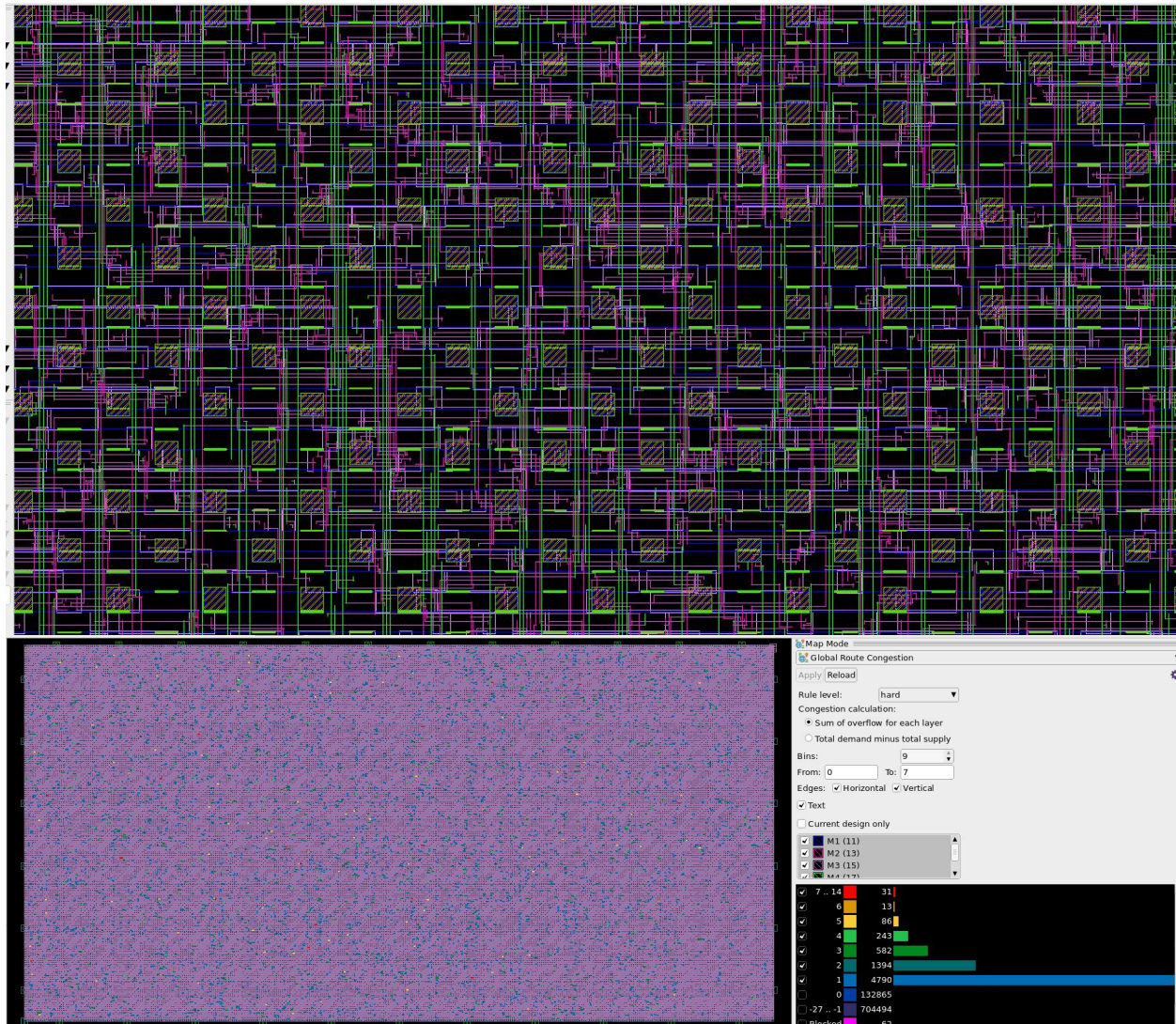
parameter	Value
Max transision	0.25
Target_skew	0.15

Compinged with CCD all Hold Violations Was Eliminated.
The Resulted Utilization after the CTS was done is 55.87% :



6.3.6 Routing

After the Routing The Utilization went up to 56.21% and solved all hold Violations.



6.3.7 Finishing

After completing routing, the design was signed off with a final set of physical and electrical verification checks:

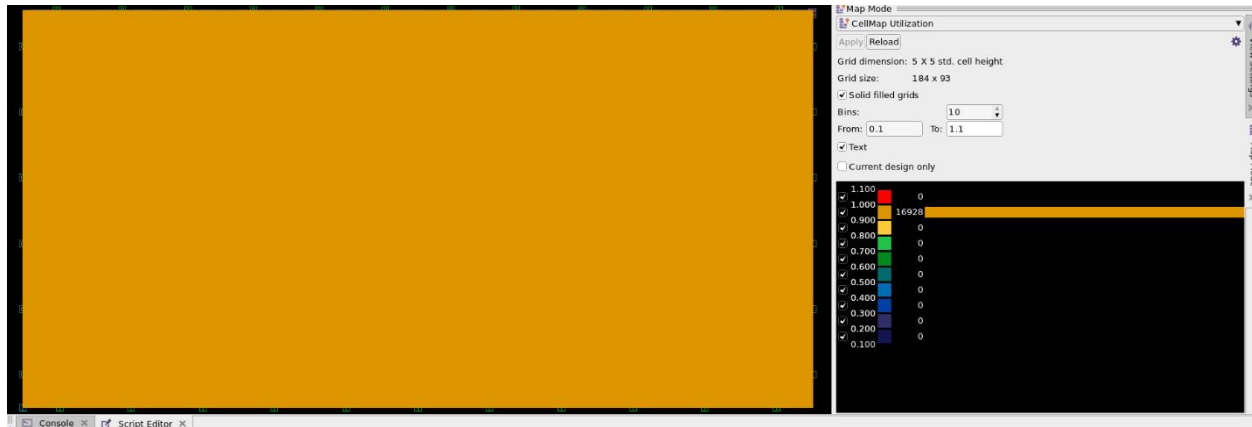
Design Rule Check (DRC): A full-chip DRC was run (excluding antenna rules), covering 276 partitions and 135,834 nets. The check reported **zero DRC violations**, with 0 open nets. Total routed wire length was **3,148,327 microns** across metal layers M1–M7 (M8/M9 unused), using 976,333 contacts/vias.

Power/Ground (PG) Verification:

- **PG DRC Check:** No errors found (completed in ~111 seconds).
- **PG Connectivity Check:** Verified across 695,895 standard cells with 849 VDD wires, 274,160 VDD vias, 893 VSS wires, and 273,797 VSS vias. Zero floating wires, vias, standard cells, or terminals were found on both VDD and VSS nets — confirming full power/ground connectivity.

LVS (Layout vs. Schematic) Check: Run across 135,838 nets with **zero short violations, zero open nets, and zero floating pins**, confirming netlist-to-layout consistency.

Legality Check: The final placement legality check reported **0 total violations** across all categories (overlaps, site alignment, orientation, spacing rules, layer rules, region violations, CTS margin violations, and coloring), confirming a fully legal placement.



Timing (Global Timing / QoR Summary):

- **Setup:** No setup violations found (WNS = 22.46 ns, TNS = 0.00).
- **Hold:** WNS = -0.00 ns, TNS = -0.03 ns, with 174 hold violations remaining (reg-to-reg only) — an extremely small residual after CTS/CCD-based hold fixing.
- All hold Violations Were Solved Using Prime Time giving TNS = -0.00 ns

Utilization: Final utilization after routing was **56.21%**, based on a total core area of 1,173,058.56 μm^2 and total cell area of 659,422.10 μm^2 .

Summary: The design achieved clean DRC, PG connectivity, PG DRC, and LVS results with zero violations, full legal placement, and timing closure with no setup violations and only negligible residual hold slack ($TNS = -0.03$ ns), confirming the design is ready for tape-out.
